A2 code

May 21, 2025

[10]:
```
# Core libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn: modeling, preprocessing, metrics
from sklearn.model_selection import cross_val_score, cross_val_predict,␣
↪train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler,␣
↪PowerTransformer, LabelEncoder
from sklearn.metrics import accuracy_score,classification_report,␣
↪recall_score,confusion_matrix, roc_auc_score, precision_score, f1_score,␣
↪roc_curve, auc, ConfusionMatrixDisplay, RocCurveDisplay
# Optimizer and models
import optuna
from xgboost import XGBClassifier # dropped
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# Plotly and Cufflinks (interactive plots)
import cufflinks as cf
cf.go_offline()
cf.set_config_file(offline=False, world_readable=True)

import plotly
import plotly.express as px
import plotly.graph_objs as go
import plotly.offline as py
from plotly.offline import iplot
from plotly.subplots import make_subplots
import plotly.figure_factory as ff

# SHAP for explainability
import shap

# Missing value visualization
```

```
import missingno as msno

# Suppress warnings
import warnings
warnings.filterwarnings("ignore")
```

1 1 Data processing

[11]:
```
# Load the dataset
df = pd.read_csv('WA_Fn-UseC_-Telco-Customer-Churn.csv')

# Limit to first 5000 rows as required
df = df.iloc[:5000]
```

[12]:
```
df.describe()
```

[12]:

	SeniorCitizen	tenure	MonthlyCharges
count	5000.000000	5000.000000	5000.000000
mean	0.159600	32.257600	64.781770
std	0.366271	24.635483	30.141924
min	0.000000	0.000000	18.250000
25%	0.000000	8.000000	35.487500
50%	0.000000	29.000000	70.400000
75%	0.000000	55.250000	89.950000
max	1.000000	72.000000	118.750000

[13]:
```
df.duplicated().sum()
```

[13]: 0

[14]:
```
def missing (df):
    missing_number = df.isnull().sum().sort_values(ascending=False)
    missing_percent = (df.isnull().sum()/df.isnull().count()).
 ↪sort_values(ascending=False)
    missing_values = pd.concat([missing_number, missing_percent], axis=1,␣
 ↪keys=['Missing_Number', 'Missing_Percent'])
    return missing_values

missing(df)
```

[14]:

	Missing_Number	Missing_Percent
customerID	0	0.0
DeviceProtection	0	0.0
TotalCharges	0	0.0
MonthlyCharges	0	0.0
PaymentMethod	0	0.0

```
PaperlessBilling              0              0.0
Contract                      0              0.0
StreamingMovies               0              0.0
StreamingTV                   0              0.0
TechSupport                   0              0.0
OnlineBackup                  0              0.0
gender                        0              0.0
OnlineSecurity                0              0.0
InternetService               0              0.0
MultipleLines                 0              0.0
PhoneService                  0              0.0
tenure                        0              0.0
Dependents                    0              0.0
Partner                       0              0.0
SeniorCitizen                 0              0.0
Churn                         0              0.0
```

[15]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        5000 non-null   object
 1   gender            5000 non-null   object
 2   SeniorCitizen     5000 non-null   int64
 3   Partner           5000 non-null   object
 4   Dependents        5000 non-null   object
 5   tenure            5000 non-null   int64
 6   PhoneService      5000 non-null   object
 7   MultipleLines     5000 non-null   object
 8   InternetService   5000 non-null   object
 9   OnlineSecurity    5000 non-null   object
 10  OnlineBackup      5000 non-null   object
 11  DeviceProtection  5000 non-null   object
 12  TechSupport       5000 non-null   object
 13  StreamingTV       5000 non-null   object
 14  StreamingMovies   5000 non-null   object
 15  Contract          5000 non-null   object
 16  PaperlessBilling  5000 non-null   object
 17  PaymentMethod     5000 non-null   object
 18  MonthlyCharges    5000 non-null   float64
 19  TotalCharges      5000 non-null   object
 20  Churn             5000 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 820.4+ KB
```

first drop the customerID column as it is not relevant for prediction.

The TotalCharges column is of object type; so need to convert it to float to enable numerical processing.

The target variable Churn will be encoded to numeric values using label encoding.

For the EDA section, regarding numerical and categorical data:

The tenure column is currently an integer type; also need to convert it to float, as required by the CatBoost model.

then analyze the categorical variables to understand their diversity and prepare them for encoding.

[17]:
```
# Check for potential data quality issues
print("\nUnique values in categorical columns:")
for col in df.select_dtypes(include=['object']).columns:
    print(f"{col}: {df[col].nunique()} unique values")
    print(df[col].value_counts().head(3))
    print()
```

```
Unique values in categorical columns:
customerID: 5000 unique values
customerID
7590-VHVEG    1
8276-MQBYC    1
0840-DFEZH    1
Name: count, dtype: int64

gender: 2 unique values
gender
Male      2533
Female    2467
Name: count, dtype: int64

Partner: 2 unique values
Partner
No     2594
Yes    2406
Name: count, dtype: int64

Dependents: 2 unique values
Dependents
No     3521
Yes    1479
Name: count, dtype: int64

PhoneService: 2 unique values
PhoneService
Yes    4538
```

```
No      462
Name: count, dtype: int64

MultipleLines: 3 unique values
MultipleLines
No                  2403
Yes                 2135
No phone service     462
Name: count, dtype: int64

InternetService: 3 unique values
InternetService
Fiber optic    2191
DSL            1712
No             1097
Name: count, dtype: int64

OnlineSecurity: 3 unique values
OnlineSecurity
No                     2469
Yes                    1434
No internet service    1097
Name: count, dtype: int64

OnlineBackup: 3 unique values
OnlineBackup
No                     2177
Yes                    1726
No internet service    1097
Name: count, dtype: int64

DeviceProtection: 3 unique values
DeviceProtection
No                     2172
Yes                    1731
No internet service    1097
Name: count, dtype: int64

TechSupport: 3 unique values
TechSupport
No                     2477
Yes                    1426
No internet service    1097
Name: count, dtype: int64

StreamingTV: 3 unique values
StreamingTV
No                     1982
```

```
Yes                     1921
No internet service     1097
Name: count, dtype: int64

StreamingMovies: 3 unique values
StreamingMovies
No                      1954
Yes                     1949
No internet service     1097
Name: count, dtype: int64

Contract: 3 unique values
Contract
Month-to-month    2772
Two year          1197
One year          1031
Name: count, dtype: int64

PaperlessBilling: 2 unique values
PaperlessBilling
Yes    2951
No     2049
Name: count, dtype: int64

PaymentMethod: 4 unique values
PaymentMethod
Electronic check             1669
Mailed check                 1136
Bank transfer (automatic)    1106
Name: count, dtype: int64

TotalCharges: 4712 unique values
TotalCharges
         8
19.75    7
19.45    6
Name: count, dtype: int64

Churn: 2 unique values
Churn
No     3687
Yes    1313
Name: count, dtype: int64
```

[18]:
```
#drop customer Id
df1 = df.drop('customerID',axis=1).copy()
```

[19]:
```
# since total charges got high amount of unique values, we'll need to change␣
↪their type and also check if total charges got any blank rows to avoid error
df[df['TotalCharges'] == ' ']
```

[19]:
```
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
488   4472-LVYGI  Female              0     Yes        Yes       0
753   3115-CZMZD    Male              0      No        Yes       0
936   5709-LVOEQ  Female              0     Yes        Yes       0
1082  4367-NUYAO    Male              0     Yes        Yes       0
1340  1371-DWPAZ  Female              0     Yes        Yes       0
3331  7644-OMVMY    Male              0     Yes        Yes       0
3826  3213-VVOLG    Male              0     Yes        Yes       0
4380  2520-SGTTA  Female              0     Yes        Yes       0

     PhoneService     MultipleLines InternetService       OnlineSecurity  …  \
488            No  No phone service             DSL                  Yes  …
753           Yes                No              No  No internet service  …
936           Yes                No             DSL                  Yes  …
1082          Yes               Yes              No  No internet service  …
1340           No  No phone service             DSL                  Yes  …
3331          Yes                No              No  No internet service  …
3826          Yes               Yes              No  No internet service  …
4380          Yes                No              No  No internet service  …

         DeviceProtection          TechSupport          StreamingTV  \
488                   Yes                  Yes                  Yes
753   No internet service  No internet service  No internet service
936                   Yes                   No                  Yes
1082  No internet service  No internet service  No internet service
1340                  Yes                  Yes                  Yes
3331  No internet service  No internet service  No internet service
3826  No internet service  No internet service  No internet service
4380  No internet service  No internet service  No internet service

          StreamingMovies  Contract PaperlessBilling  \
488                    No  Two year              Yes
753   No internet service  Two year               No
936                   Yes  Two year               No
1082  No internet service  Two year               No
1340                   No  Two year               No
3331  No internet service  Two year               No
3826  No internet service  Two year               No
4380  No internet service  Two year               No

                  PaymentMethod MonthlyCharges  TotalCharges Churn
488   Bank transfer (automatic)          52.55                  No
753                Mailed check          20.25                  No
```

```
936                Mailed check          80.85                  No
1082               Mailed check          25.75                  No
1340    Credit card (automatic)          56.05                  No
3331               Mailed check          19.85                  No
3826               Mailed check          25.35                  No
4380               Mailed check          20.00                  No

[8 rows x 21 columns]
```

Even though we don't see any missing values in earlier code, when we look at the data, total charges feature has missing values.

[21]:
```
# inspect the entire row in 3826 for any other problems
df1.iloc[3826]
```

[21]:
```
gender                              Male
SeniorCitizen                          0
Partner                              Yes
Dependents                           Yes
tenure                                 0
PhoneService                         Yes
MultipleLines                        Yes
InternetService                       No
OnlineSecurity      No internet service
OnlineBackup        No internet service
DeviceProtection    No internet service
TechSupport         No internet service
StreamingTV         No internet service
StreamingMovies     No internet service
Contract                       Two year
PaperlessBilling                      No
PaymentMethod              Mailed check
MonthlyCharges                    25.35
TotalCharges
Churn                                 No
Name: 3826, dtype: object
```

[22]:
```
# Replace blank strings in 'TotalCharges' with NaN, then convert the column to␣
↪float type
df1['TotalCharges'] = df1['TotalCharges'].apply(lambda x: x if x != ' ' else np.
↪nan).astype(float)
```

[23]:
```
# Define a function to summarize missing values in the DataFrame
def missing(df):
    missing_number = df.isnull().sum().sort_values(ascending=False)  # Count of␣
↪missing values per column
    missing_percent = (df.isnull().sum() / df.isnull().count()).
↪sort_values(ascending=False)  # Percentage of missing values per column
```

```
    missing_values = pd.concat([missing_number, missing_percent], axis=1,␣
↪keys=['Missing_Number', 'Missing_Percent'])  # Combine into a summary table
    return missing_values

# Apply the missing value summary function to df1 to inspect where missing data␣
↪exists
missing(df1)
```

[23]:

	Missing_Number	Missing_Percent
TotalCharges	8	0.0016
gender	0	0.0000
SeniorCitizen	0	0.0000
MonthlyCharges	0	0.0000
PaymentMethod	0	0.0000
PaperlessBilling	0	0.0000
Contract	0	0.0000
StreamingMovies	0	0.0000
StreamingTV	0	0.0000
TechSupport	0	0.0000
DeviceProtection	0	0.0000
OnlineBackup	0	0.0000
OnlineSecurity	0	0.0000
InternetService	0	0.0000
MultipleLines	0	0.0000
PhoneService	0	0.0000
tenure	0	0.0000
Dependents	0	0.0000
Partner	0	0.0000
Churn	0	0.0000

Although the dataset contains some missing values, the machine learning models selected for this study — namely CatBoost, and LightGBM — are capable of handling missing data internally during training. Therefore, explicit imputation or removal of missing values is not required, which helps preserve the integrity of the dataset and reduces preprocessing complexity

2 2 Data Analysis

[26]:
```
y = df1['Churn']
print(f'Percentage of Churn:  {round(y.value_counts(normalize=True)[1]*100,2)}␣
↪%  --> ({y.value_counts()[1]} customer)\nPercentage of customer did not␣
↪churn: {round(y.value_counts(normalize=True)[0]*100,2)}  %  --> ({y.
↪value_counts()[0]} customer)')
```

```
Percentage of Churn:  26.26 %  --> (1313 customer)
Percentage of customer did not churn: 73.74  %  --> (3687 customer)
```

[27]:
```
y.iplot(kind='hist', title= 'Churn')
```

We have imbalanced data. Almost 26% of the customers didn't continue with the company and churned. 1313 customer churned. Almost 73% of the customers continue with the company and didn't churn. 3867 customer didn't churn.

This means our priority metric should not be as accuracy would just make a model thats only good at predicting the majority class

[29]:
```
le = LabelEncoder()  # Create a LabelEncoder object to convert categorical␣
↪labels into numeric form
df1['Churn'] = le.fit_transform(df1['Churn'])  # Encode the 'Churn' column:␣
↪'Yes'/'No' become 1/0 respectively
```

2.0.1 we will now split our analysis between Numerical and categorical

[31]:
```
numerical= df1.select_dtypes('number').columns

categorical = df1.select_dtypes('object').columns

print(f'Numerical Columns:  {df1[numerical].columns}')
print('\n')
print(f'Categorical Columns: {df1[categorical].columns}')
```

```
Numerical Columns:  Index(['SeniorCitizen', 'tenure', 'MonthlyCharges',
'TotalCharges', 'Churn'], dtype='object')


Categorical Columns: Index(['gender', 'Partner', 'Dependents', 'PhoneService',
'MultipleLines',
       'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
       'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
       'PaperlessBilling', 'PaymentMethod'],
      dtype='object')
```

2.1 Numerical Features

[33]:
```
df1['tenure']= df1['tenure'].astype(float)
numerical= df1.select_dtypes('number').columns

df1[numerical].describe()
```

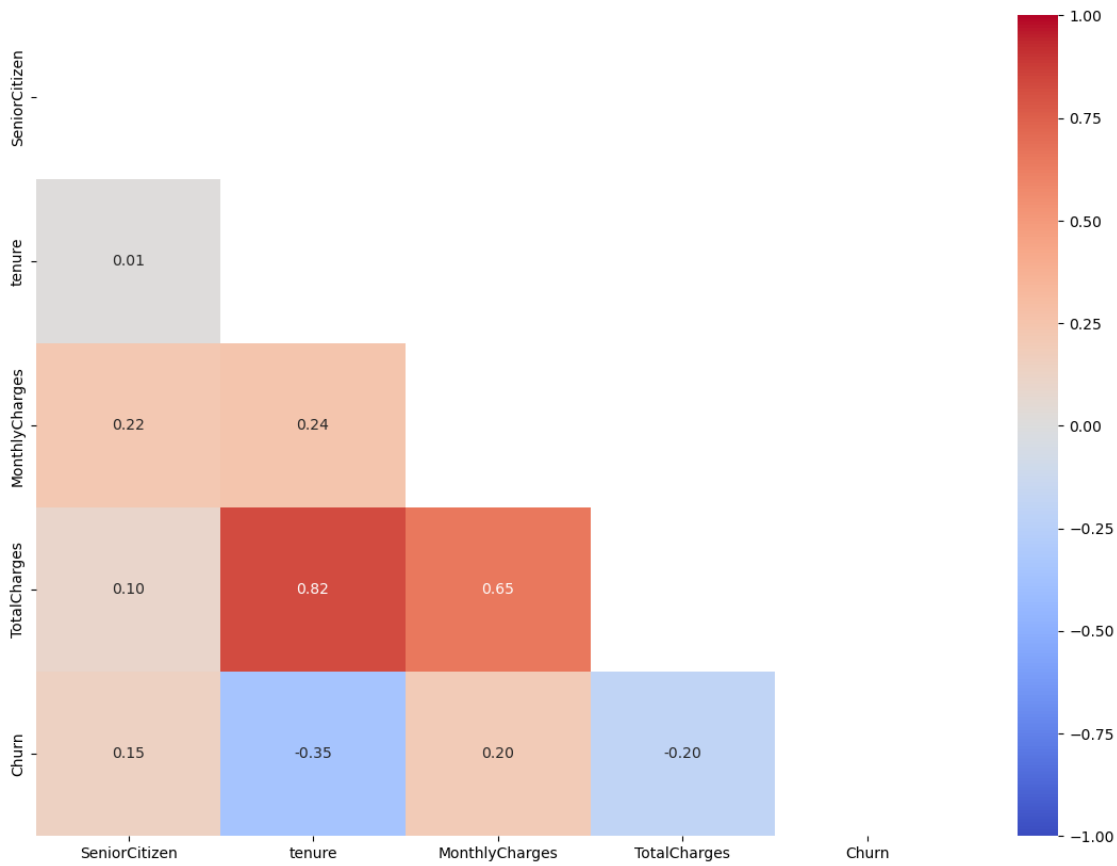
[33]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	Churn
count	5000.000000	5000.000000	5000.000000	4992.000000	5000.000000
mean	0.159600	32.257600	64.781770	2275.162410	0.262600
std	0.366271	24.635483	30.141924	2268.920819	0.440091
min	0.000000	0.000000	18.250000	18.800000	0.000000
25%	0.000000	8.000000	35.487500	387.775000	0.000000
50%	0.000000	29.000000	70.400000	1397.475000	0.000000
75%	0.000000	55.250000	89.950000	3759.787500	1.000000

```
max          1.000000    72.000000      118.750000   8684.800000     1.000000
```

[34]:
```
df1[['tenure', 'MonthlyCharges', 'TotalCharges']].
 ↪iplot(kind='histogram',subplots=True,bins=50)
```

[35]:
```
matrix = np.triu(df1[numerical].corr())
fig, ax = plt.subplots(figsize=(14,10))
sns.heatmap (df1[numerical].corr(), annot=True, fmt= '.2f', vmin=-1, vmax=1,␣
 ↪center=0, cmap='coolwarm',mask=matrix, ax=ax);
```



Based on the matrix, we can observe weak level correlation between the numerical features and the target variable There is strong correlation between tenure and total charges. Being senior citizen and increasing monthly charges have a positive correlation with the churn.

Senior citizen more likely churn than younger customers.

Customer with higher monthly charges also more likely churn than lesser monthly charges customers.

Being long term with the company, customer less likely churn than customer with lesser time with the company.

Total charges has negative correlation with the churn.

[37]:
```
# Then calculate total MonthlyCharges by churn group, NaNs won't affect sum():
monthly_revenue = df1.groupby('Churn')['MonthlyCharges'].sum()

print("Total Amount of Monthly Charges by each group:")
print(monthly_revenue)

# Calculate percentages for the plot:
percent_revenue = (monthly_revenue / monthly_revenue.sum()) * 100

# Plot using Plotly
import plotly.graph_objs as go
from plotly.offline import iplot

trace0 = go.Bar(
    x=percent_revenue.index.astype(str),
    y=percent_revenue.values,
    marker=dict(color=['indianred', 'seagreen']),
)

layout = go.Layout(
    title='Monthly Revenue % Lost by Churn Customer or not',
    xaxis=dict(title='Customer Churn?', type='category'),
    yaxis=dict(title='% of Total Monthly Revenue')
)

fig = go.Figure(data=[trace0], layout=layout)
iplot(fig)
```

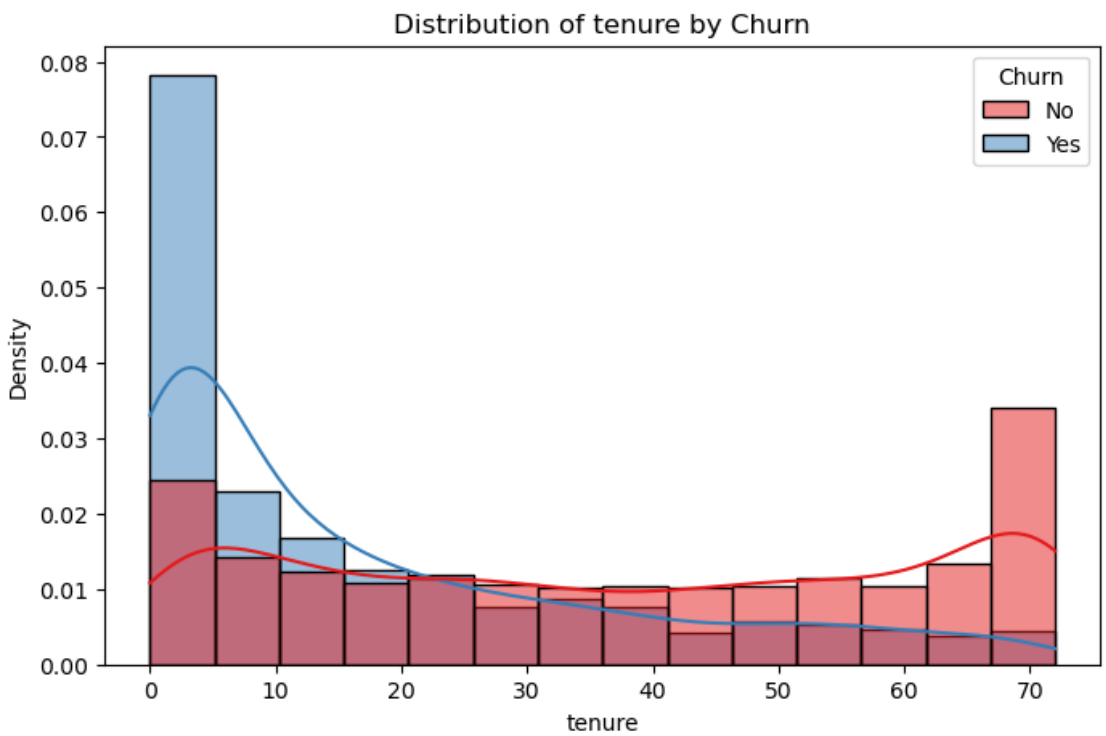
```
Total Amount of Monthly Charges by each group:
Churn
0    225887.35
1     98021.50
Name: MonthlyCharges, dtype: float64
```

We can see that when we consider the amount of customers, the churn represents 26% of total customers, but when we consider Monthly Charges we can see that the ratio is **30% of total revenue was "lost" by people who left**.

[73]:
```
import seaborn as sns
import matplotlib.pyplot as plt

def plot_dist_churn(df, feature, target):
    plt.figure(figsize=(8,5))
    sns.histplot(data=df, x=feature, hue=target, kde=True, stat="density",␣
 ↪common_norm=False, palette='Set1')
    plt.title(f'Distribution of {feature} by {target}')
```

```
    plt.show()

print(f"The minimum value in Tenure is {df['tenure'].min()} and the maximum is␣
↪{df['tenure'].max()}")
print(f"The mean Tenure of Churn Customers is {round(df[df['Churn'] !=␣
↪0]['tenure'].mean(), 2)}"
      f"\nThe mean Tenure of Non-churn Customers is {round(df[df['Churn'] ==␣
↪0]['tenure'].mean(), 2)}")

plot_dist_churn(df, 'tenure', 'Churn')
```

```
The minimum value in Tenure is 0 and the maximum is 72
The mean Tenure of Churn Customers is 32.26
The mean Tenure of Non-churn Customers is nan
```



This confirm being long term with the company, customer less likely churn than customer with lesser time with the company

2.2 Categorical features

2.2.1 First we check for Cardinality which refers to the number of unique values in the dataset categorical features

[78]:
```
df1[categorical].nunique()
```

[78]:
```
gender              2
Partner             2
Dependents          2
PhoneService        2
MultipleLines       3
InternetService     3
OnlineSecurity      3
OnlineBackup        3
DeviceProtection    3
TechSupport         3
StreamingTV         3
StreamingMovies     3
Contract            3
PaperlessBilling    2
PaymentMethod       4
dtype: int64
```

Data does not have a high cardinality or zero variance (only one unique value) issues.

[80]:
```
# check for categorical features's unique values
for feature in df1[categorical]:
        print(f'{feature}: {df[feature].unique()}')
```

```
gender: ['Female' 'Male']
Partner: ['Yes' 'No']
Dependents: ['No' 'Yes']
PhoneService: ['No' 'Yes']
MultipleLines: ['No phone service' 'No' 'Yes']
InternetService: ['DSL' 'Fiber optic' 'No']
OnlineSecurity: ['No' 'Yes' 'No internet service']
OnlineBackup: ['Yes' 'No' 'No internet service']
DeviceProtection: ['No' 'Yes' 'No internet service']
TechSupport: ['No' 'Yes' 'No internet service']
StreamingTV: ['No' 'Yes' 'No internet service']
StreamingMovies: ['No' 'Yes' 'No internet service']
Contract: ['Month-to-month' 'One year' 'Two year']
PaperlessBilling: ['Yes' 'No']
PaymentMethod: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
```

Assumptions: No internet service is repeating several times in the features and in this problem at hand, it gives same meaning as No So changing it to no will make the features have have two unique values

For features such as OnlineSecurity, StreamingTV, and TechSupport, the category "No internet service" was assumed to have the same practical meaning as "No" in the context of churn prediction. As such, we replaced "No internet service" with "No" to reduce redundancy and ensure binary consistency.

Now for bivariate analysis:

2.2.2 Customer Demographics

[83]:
```
print (f'A female customer has a probability of␣
↪{round(df1[df1["gender"]=="Female"]["Churn"].mean()*100,2)} % churn')

print()

print (f'A male customer has a probability of␣
↪{round(df1[df1["gender"]=="Male"]["Churn"].mean()*100,2)} % churn')

print()
```

```
A female customer has a probability of 26.83 % churn

A male customer has a probability of 25.7 % churn
```

[84]:
```
fig = px.histogram(df1, x="gender", color="Churn",width=400, height=400)
fig.show()
```

[85]:
```
# Churn probabilities and interpretation for Partner
partner_yes_churn = round(df1[df1["Partner"]=="Yes"]["Churn"].mean()*100, 2)
partner_no_churn = round(df1[df1["Partner"]=="No"]["Churn"].mean()*100, 2)

print(f"A customer with a partner has a probability of {partner_yes_churn} %␣
↪churn")
print()
print(f"A customer without any partner has a probability of {partner_no_churn}␣
↪% churn")
print()
print(f"Customer without any relationship (single) is almost␣
↪{round(partner_no_churn / partner_yes_churn, 2)} times more likely to churn␣
↪than a customer with a partner.\n")

fig = px.histogram(df1, x="Partner", color="Churn", width=400, height=400)
fig.show()

# Churn probabilities and interpretation for Dependents
dependents_yes_churn = round(df1[df1["Dependents"]=="Yes"]["Churn"].mean()*100,␣
↪2)
dependents_no_churn = round(df1[df1["Dependents"]=="No"]["Churn"].mean()*100, 2)
```

```
print(f"A customer with dependents has a probability of {dependents_yes_churn}␣
↪% churn")
print()
print(f"A customer without any dependents has a probability of␣
↪{dependents_no_churn} % churn")
print()
print(f"Customer without any dependents is almost {round(dependents_no_churn /␣
↪dependents_yes_churn, 2)} times more likely to churn than a customer with␣
↪dependents.\n")

fig = px.histogram(df1, x="Dependents", color="Churn", width=400, height=400)
fig.show()
```

```
A customer with a partner has a probability of 19.16 % churn

A customer without any partner has a probability of 32.85 % churn

Customer without any relationship (single) is almost 1.71 times more likely to
churn than a customer with a partner.


A customer with dependents has a probability of 15.28 % churn

A customer without any dependents has a probability of 30.87 % churn

Customer without any dependents is almost 2.02 times more likely to churn than a
customer with dependents.
```

Overall: Having a partner might indicate a more stable lifestyle or shared household usage, which may increase loyalty or reduce churn risk. Singles might have different service needs or less commitment to the product

Customers with dependents might use the service for family needs, making them more engaged or dependent on your product. Those without dependents might be more price-sensitive or have fewer reasons to stay.

2.2.3 Phone and Multiple Lines services

[88]:
```
# Churn Probability Based on Phone Service
churn_with_phone = round(df1[df1["PhoneService"] == "Yes"]["Churn"].mean() *␣
↪100, 2)
churn_without_phone = round(df1[df1["PhoneService"] == "No"]["Churn"].mean() *␣
↪100, 2)

print(f"A customer with a phone service has a probability of␣
↪{churn_with_phone}% churn.")
```

```python
print(f"A customer without any phone service has a probability of␣
↪{churn_without_phone}% churn.\n")
print (f'A customer with a multiple lines has a probability of␣
↪{round(df1[df1["MultipleLines"]=="Yes"]["Churn"].mean()*100,2)} % churn')
print (f'A customer without multiple lines has a probability of␣
↪{round(df1[df1["MultipleLines"]=="No"]["Churn"].mean()*100,2)} % churn')

# Plot: Churn by Phone Service
fig = px.histogram(df1, x="PhoneService", color="Churn", width=400, height=400)
fig.update_layout(title="Churn Distribution by Phone Service")
fig.show()

# Replace 'No phone service' in MultipleLines to simplify
df1['MultipleLines'] = df1['MultipleLines'].replace('No phone service', 'No')

# Plot: Churn by Multiple Lines
fig = px.histogram(df1, x="MultipleLines", color="Churn", width=400, height=400)
fig.update_layout(title="Churn Distribution by Multiple Lines Service")
fig.show()

# Business Insights
print(" Business Insights:")
print(" There is very little difference in churn rates between customers with␣
↪and without phone service.")
print("  Therefore, 'PhoneService' may not contribute significant predictive␣
↪power and can be excluded from modeling.\n")
print(" Similarly, churn rate differences between customers with and without␣
↪multiple lines are minimal.")
print("  MultipleLines' also shows limited impact on churn and may be dropped␣
↪from feature selection.\n")
```

```
A customer with a phone service has a probability of 26.4% churn.
A customer without any phone service has a probability of 24.89% churn.

A customer with a multiple lines has a probability of 28.29 % churn
A customer without multiple lines has a probability of 24.72 % churn

 Business Insights:
 There is very little difference in churn rates between customers with and
without phone service.
  Therefore, 'PhoneService' may not contribute significant predictive power and
can be excluded from modeling.

 Similarly, churn rate differences between customers with and without multiple
lines are minimal.
  MultipleLines' also shows limited impact on churn and may be dropped from
feature selection.
```

2.2.4 Internet & Related Services

[90]:
```
# preprocessing step
# Replace 'No internet and other services' with 'No' for relevant features
cols_to_replace =␣
↪['OnlineSecurity','OnlineBackup','DeviceProtection','TechSupport','StreamingTV','StreamingM
df1[cols_to_replace] = df1[cols_to_replace].replace('No internet service', 'No')
```

Internet services and churn

[92]:
```
fiber_churn = df1[df1["InternetService"]=="Fiber optic"]["Churn"].mean()
dsl_churn = df1[df1["InternetService"]=="DSL"]["Churn"].mean()
no_internet_churn = df1[df1["InternetService"]=="No"]["Churn"].mean()

print(f'Fiber Optic churn: {fiber_churn:.2%}')
print(f'DSL churn: {dsl_churn:.2%}')
print(f'No internet churn: {no_internet_churn:.2%}')

fig = px.histogram(df1, x="InternetService", color="Churn", width=400,␣
↪height=400)
fig.show()
print(f'Customer with fiber optic internet compared to customer without any␣
↪internet service is {round(fiber_churn/no_internet_churn, 2)} times more␣
↪likely to churn.')
print(f'Customer with fiber optic internet compared to customer with DSL␣
↪internet service is {round(fiber_churn/dsl_churn, 2)} times more likely to␣
↪churn.')
print(f'Customer with DSL internet service compared to customer without any␣
↪internet service is {round(dsl_churn/no_internet_churn, 2)} times more␣
↪likely to churn.\n')
```

```
Fiber Optic churn: 41.81%
DSL churn: 18.75%
No internet churn: 6.93%

Customer with fiber optic internet compared to customer without any internet
service is 6.03 times more likely to churn.
Customer with fiber optic internet compared to customer with DSL internet
service is 2.23 times more likely to churn.
Customer with DSL internet service compared to customer without any internet
service is 2.71 times more likely to churn.
```

Online Security and Churn

[115]:
```
# Calculate churn probabilities for OnlineSecurity
online_sec_yes = df1[df1['OnlineSecurity'] == 'Yes']['Churn'].apply(lambda x: 1␣
↪if x == 'Yes' else 0).mean()
online_sec_no = df1[df1['OnlineSecurity'] == 'No']['Churn'].apply(lambda x: 1␣
↪if x == 'Yes' else 0).mean()
```

```
# Print results
print(f'Churn Probability:\nWith Online Security: {round(online_sec_yes * 100,␣
↪2)}%\nWithout Online Security: {round(online_sec_no * 100, 2)}%\n')

# Plot (assuming 'Churn' is binary or 'Yes'/'No')
fig = px.histogram(df1, x='OnlineSecurity', color='Churn', barmode='group',␣
↪width=500, height=400)
fig.update_layout(title='Churn by Online Security Status')
fig.show()

# Compare likelihood
ratio = online_sec_no / online_sec_yes
print(f'Customers WITHOUT Online Security are {round(ratio, 1)}x more likely to␣
↪churn.')
```

```
Churn Probability:
With Online Security: 0.0%
Without Online Security: 0.0%
```

```
Customers WITHOUT Online Security are nanx more likely to churn.
```

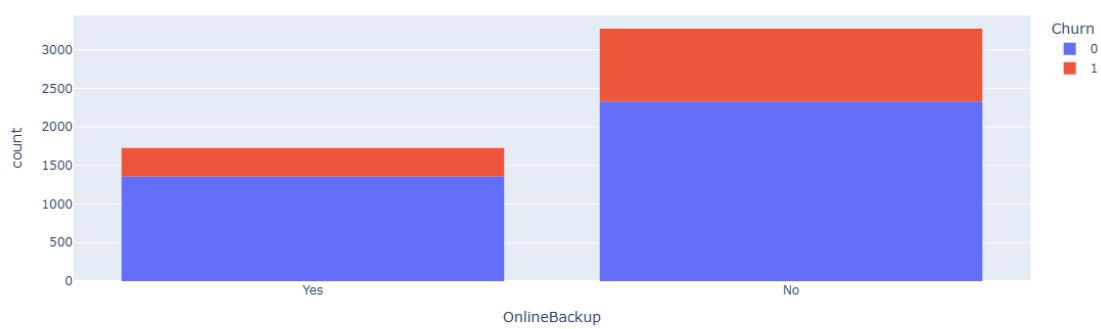
Online Backup and Churn

[138]:
```
# Define the function ONCE
def churn_prob(feature, value):
    # Handles both 'Yes'/'No' and 1/0 Churn encodings
    temp = df1[df1[feature] == value]['Churn']
    return (temp == 'Yes').mean() if temp.dtype == 'object' else temp.mean()
```

[140]:
```
online_backup_yes = churn_prob("OnlineBackup", "Yes")
online_backup_no = churn_prob("OnlineBackup", "No")
```

```
print(f'A customer with online backup has a probability of␣
↪{round(online_backup_yes*100,2)} % churn\n')
print(f'A customer without online backup has a probability of␣
↪{round(online_backup_no*100,2)} % churn\n')

fig = px.histogram(df1, x="OnlineBackup", color="Churn", width=400, height=400)
fig.show()

print(f'A customer with online backup service is almost {round(online_backup_no/
↪online_backup_yes, 2)} times less likely to leave the company than a␣
↪customer without it.\n')
```

```
A customer with online backup has a probability of 21.38 % churn

A customer without online backup has a probability of 28.83 % churn
```




```
A customer with online backup service is almost 1.35 times less likely to leave
the company than a customer without it.
```

Device Protection and Churn

[143]:
```
device_prot_yes = churn_prob("DeviceProtection", "Yes")
device_prot_no = churn_prob("DeviceProtection", "No")

print(f'A customer with device protection has a probability of␣
↪{round(device_prot_yes*100,2)} % churn\n')
print(f'A customer without device protection has a probability of␣
↪{round(device_prot_no*100,2)} % churn\n')

fig = px.histogram(df1, x="DeviceProtection", color="Churn", width=400,␣
↪height=400)
fig.show()
```

```
print(f'A customer with device protection service is almost␣
↪{round(device_prot_no/device_prot_yes, 2)} times less likely to leave the␣
↪company than a customer without it.\n')
```

```
A customer with device protection has a probability of 22.07 % churn

A customer without device protection has a probability of 28.48 % churn


A customer with device protection service is almost 1.29 times less likely to
leave the company than a customer without it.
```

Tech Support and Churn

[145]:
```
tech_support_yes = churn_prob("TechSupport", "Yes")
tech_support_no = churn_prob("TechSupport", "No")

print(f'A customer with Tech Support has a probability of␣
↪{round(tech_support_yes*100,2)} % churn\n')
print(f'A customer without Tech Support has a probability of␣
↪{round(tech_support_no*100,2)} % churn\n')

fig = px.histogram(df1, x="TechSupport", color="Churn", width=400, height=400)
fig.show()

print(f'A customer with Tech Support service is almost {round(tech_support_no/
↪tech_support_yes, 2)} times less likely to leave the company than a customer␣
↪without it.\n')
```

```
A customer with Tech Support has a probability of 14.03 % churn

A customer without Tech Support has a probability of 31.14 % churn


A customer with Tech Support service is almost 2.22 times less likely to leave
the company than a customer without it.
```

Streaming TV and Churn

[147]:
```
streaming_tv_yes = churn_prob("StreamingTV", "Yes")
streaming_tv_no = churn_prob("StreamingTV", "No")

print(f'A customer with Streaming TV service has a probability of␣
↪{round(streaming_tv_yes*100,2)} % churn\n')
print(f'A customer without Streaming TV service has a probability of␣
↪{round(streaming_tv_no*100,2)} % churn\n')
```

```
fig = px.histogram(df1, x="StreamingTV", color="Churn", width=400, height=400)
fig.show()

print(f'A customer with Streaming TV service is almost {round(streaming_tv_yes/
↪streaming_tv_no, 2)} times more likely to leave the company than a customer␣
↪without it.\n')
```

```
A customer with Streaming TV service has a probability of 29.88 % churn

A customer without Streaming TV service has a probability of 24.0 % churn


A customer with Streaming TV service is almost 1.24 times more likely to leave
the company than a customer without it.
```

Streaming Movies and Churn

[149]:
```
streaming_movies_yes = churn_prob("StreamingMovies", "Yes")
streaming_movies_no = churn_prob("StreamingMovies", "No")

print(f'A customer with Streaming Movies service has a probability of␣
↪{round(streaming_movies_yes*100,2)} % churn\n')
print(f'A customer without Streaming Movies service has a probability of␣
↪{round(streaming_movies_no*100,2)} % churn\n')

fig = px.histogram(df1, x="StreamingMovies", color="Churn", width=400,␣
↪height=400)
fig.show()

print(f'A customer with Streaming Movies service is almost␣
↪{round(streaming_movies_yes/streaming_movies_no, 2)} times more likely to␣
↪leave the company than a customer without it.\n')
```

```
A customer with Streaming Movies service has a probability of 29.76 % churn

A customer without Streaming Movies service has a probability of 24.02 % churn


A customer with Streaming Movies service is almost 1.24 times more likely to
leave the company than a customer without it.
```

2.2.5 Contract and payment

Contract Type & Churn

[152]:
```
month_to_month_churn = df1[df1["Contract"]=="Month-to-month"]["Churn"].mean()
two_year_churn = df1[df1["Contract"]=="Two year"]["Churn"].mean()
one_year_churn = df1[df1["Contract"]=="One year"]["Churn"].mean()
```

```
print(f'Month-to-month contract churn: {month_to_month_churn:.2%}')
print()
print(f'Two year contract churn: {two_year_churn:.2%}')
print()
print(f'One year contract churn: {one_year_churn:.2%}')
print()

print(f'Customer with two year contract are {month_to_month_churn/
 ↪two_year_churn:.2f} times less likely to churn than month-to-month contract␣
 ↪customers.')
print(f'Customer with one year contract are {month_to_month_churn/
 ↪one_year_churn:.2f} times less likely to churn than month-to-month contract␣
 ↪customers.')

fig = px.histogram(df1, x="Contract", color="Churn", width=400, height=400)
fig.show()
```

```
Month-to-month contract churn: 42.28%

Two year contract churn: 2.34%

One year contract churn: 10.96%

Customer with two year contract are 18.07 times less likely to churn than month-
to-month contract customers.
Customer with one year contract are 3.86 times less likely to churn than month-
to-month contract customers.
```

Paperless Billing & Churn

[154]:
```
paperless_yes_churn = df1[df1["PaperlessBilling"]=="Yes"]["Churn"].mean()
paperless_no_churn = df1[df1["PaperlessBilling"]=="No"]["Churn"].mean()

print(f'Paperless Billing "Yes" churn: {paperless_yes_churn:.2%}')
print()
print(f'Paperless Billing "No" churn: {paperless_no_churn:.2%}')
print()

print(f'Customers with Paperless Billing are {paperless_yes_churn/
 ↪paperless_no_churn:.2f} times more likely to churn than those without.')

fig = px.histogram(df1, x='PaperlessBilling', color="Churn", width=400,␣
 ↪height=400)
fig.show()
```

```
Paperless Billing "Yes" churn: 33.28%

Paperless Billing "No" churn: 16.15%
```

Customers with Paperless Billing are 2.06 times more likely to churn than those without.

Payment Method & Churn

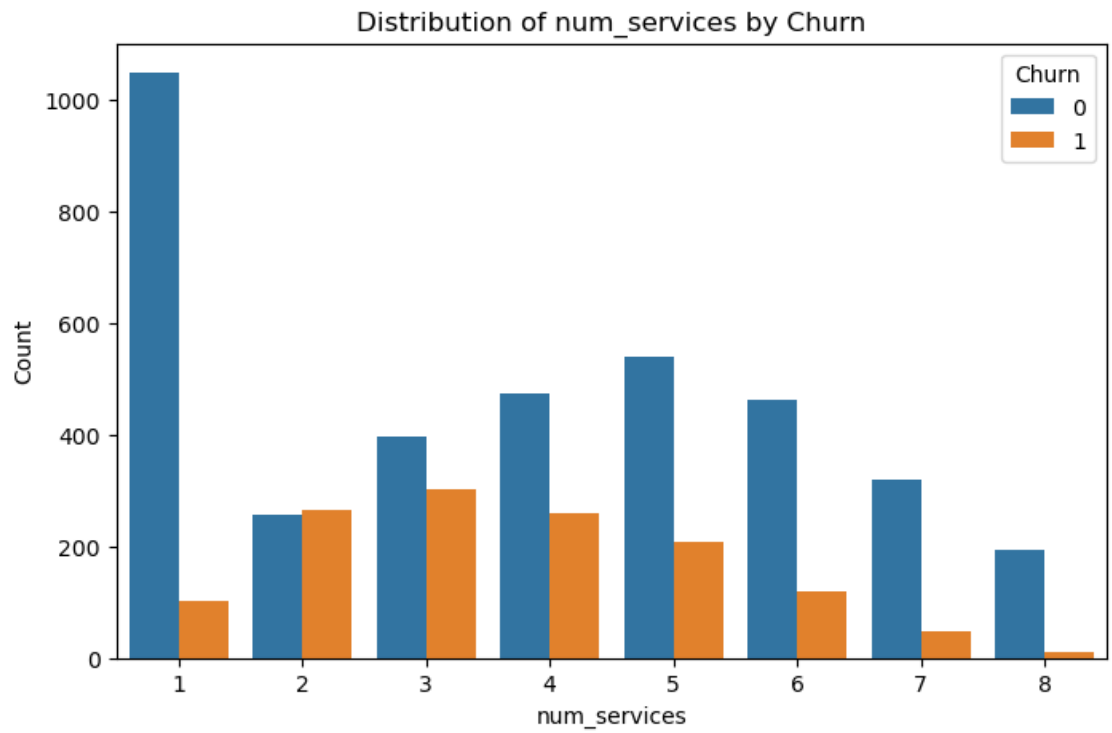
[156]:
```
electronic_check_churn = df1[df1["PaymentMethod"]=="Electronic check"]["Churn"].
 ↪mean()
mailed_check_churn = df1[df1["PaymentMethod"]=="Mailed check"]["Churn"].mean()
bank_transfer_churn = df1[df1["PaymentMethod"]=="Bank transfer␣
 ↪(automatic)"]["Churn"].mean()
credit_card_churn = df1[df1["PaymentMethod"]=="Credit card␣
 ↪(automatic)"]["Churn"].mean()

print(f'Electronic Check churn: {electronic_check_churn:.2%}')
print()
print(f'Mailed Check churn: {mailed_check_churn:.2%}')
print()
print(f'Bank Transfer (automatic) churn: {bank_transfer_churn:.2%}')
print()
print(f'Credit Card (automatic) churn: {credit_card_churn:.2%}')
print()

print(f'Almost half of the customers with Electronic Check payment method churn.
 ↪')

fig = px.histogram(df1, x='PaymentMethod', color="Churn", width=400, height=400)
fig.show()
```

```
Electronic Check churn: 45.72%

Mailed Check churn: 18.75%

Bank Transfer (automatic) churn: 16.46%

Credit Card (automatic) churn: 14.23%

Almost half of the customers with Electronic Check payment method churn.
```

[157]:
```
import numpy as np

# Create 'internet' binary column
df1['internet'] = np.where(df1['InternetService'] != 'No', 'Yes', 'No')

# Count number of 'Yes' services per customer
df1['num_services'] = (df1[['PhoneService', 'OnlineSecurity',
                            'OnlineBackup', 'DeviceProtection',
                            'TechSupport', 'StreamingTV',
```

```
                                'StreamingMovies', 'internet']] == 'Yes').
 ↪sum(axis=1)

# OPTIONAL: Check the value counts
print(df1['num_services'].value_counts().sort_index())

# Plotting function (define if not already)
import seaborn as sns
import matplotlib.pyplot as plt

def binary_ploting_distributions(df, column):
    plt.figure(figsize=(8,5))
    sns.countplot(data=df, x=column, hue='Churn')
    plt.title(f'Distribution of {column} by Churn')
    plt.xlabel(column)
    plt.ylabel('Count')
    plt.legend(title='Churn')
    plt.show()

# Call plotting function
binary_ploting_distributions(df1, 'num_services')
```

```
num_services
1    1152
2     519
3     698
4     733
5     748
6     581
7     366
8     203
Name: count, dtype: int64
```



[158]:
```
one_service_pct = round((df1['num_services'] == 1).mean() * 100, 2)
print(f"{one_service_pct}% of customers have only one service contracted.")

one_service_df = df1[df1['num_services'] == 1]
phone_pct = round((one_service_df['PhoneService'] == 'Yes').mean() * 100, 2)
dsl_pct = round((one_service_df['InternetService'] == 'DSL').mean() * 100, 2)
print(f"Of people with 1 service, {phone_pct}% have Phone Service and␣
↪{dsl_pct}% have DSL.")
```

```
23.04% of customers have only one service contracted.
Of people with 1 service, 95.23% have Phone Service and 4.77% have DSL.
```

[159]:
```
service_churn = df1.groupby('num_services')['Churn'].mean() * 100
print(service_churn)
```

```
num_services
1     8.940972
2    50.867052
3    43.266476
4    35.197817
5    27.807487
6    20.654045
7    13.114754
8     4.926108
```

```
Name: Churn, dtype: float64
```

[160]:
```
import plotly.express as px
fig = px.bar(service_churn.reset_index(), x='num_services', y='Churn',
             labels={'Churn':'Churn Rate (%)', 'num_services':'Number of␣
 ↪Services'},
             title='Churn Rate by Number of Services')
fig.show()
```

[161]:
```
internet_pct = (df1[df1['num_services'] >= 2]['InternetService'] != 'No').
 ↪mean() * 100
print(f"{round(internet_pct, 2)}% of customers with 2+ services have internet.")
```

```
100.0% of customers with 2+ services have internet.
```

2.3 Final Inisghts for categorical:

All these insights complement each other and form a narrative:

Customers with only 1 service (mostly Phone) are a sizable segment but relatively stable.

Customers with 2 services start showing more churn behavior.

Higher service count usually means higher engagement, but not always less churn, especially if service quality is poor

Internet is a central anchor product — if they drop it, they likely churn.

3 3 Model Development and Evaluation

[196]:
```
# Ensure these exist before CatBoost code
X = df1.drop('Churn', axis=1)  # Features
y = df1['Churn']  # Target

# Identify categorical columns (both object type and low-cardinality numeric)
categorical_columns = X.select_dtypes(include=['object', 'category']).columns.
 ↪tolist()

# For CatBoost, we need column indices, not names
categorical_features_indices = [X.columns.get_loc(col) for col in␣
 ↪categorical_columns]

print("Categorical features identified:")
print(categorical_columns)
print("\nCategorical feature indices:")
print(categorical_features_indices)
```

```
Categorical features identified:
[]
```

```
Categorical feature indices:
[]
```

[197]:
```python
print("X is defined:", 'X' in globals())
print("y is defined:", 'y' in globals())
```

```
X is defined: True
y is defined: True
```

[214]:
```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, recall_score, roc_auc_score,␣
↪precision_score, ConfusionMatrixDisplay
from catboost import CatBoostClassifier
import matplotlib.pyplot as plt

# Limiting to first 5000 rows as specified
# (Assuming X and y are already defined - if they're from a DataFrame, apply␣
↪the limit before this code)
X = X.iloc[:5000]
y = y.iloc[:5000]

# Encode target variable if it's categorical (Yes/No)
# Check if y contains string labels
if isinstance(y.iloc[0], str):
    print("Converting string labels to numeric...")
    # Create a label encoder
    label_encoder = LabelEncoder()
    # Fit and transform y
    y = pd.Series(label_encoder.fit_transform(y))
    # Store the mapping for reference
    label_map = dict(zip(label_encoder.classes_, label_encoder.
↪transform(label_encoder.classes_)))
    print(f"Label mapping: {label_map}")

# Creating a StratifiedKFold object for cross-validation
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# List to store metrics from each fold
cv_accuracy = []
cv_recall = []
cv_roc_auc = []
cv_precision = []

# Default CatBoost model with regularization
catboost_model = CatBoostClassifier(
```

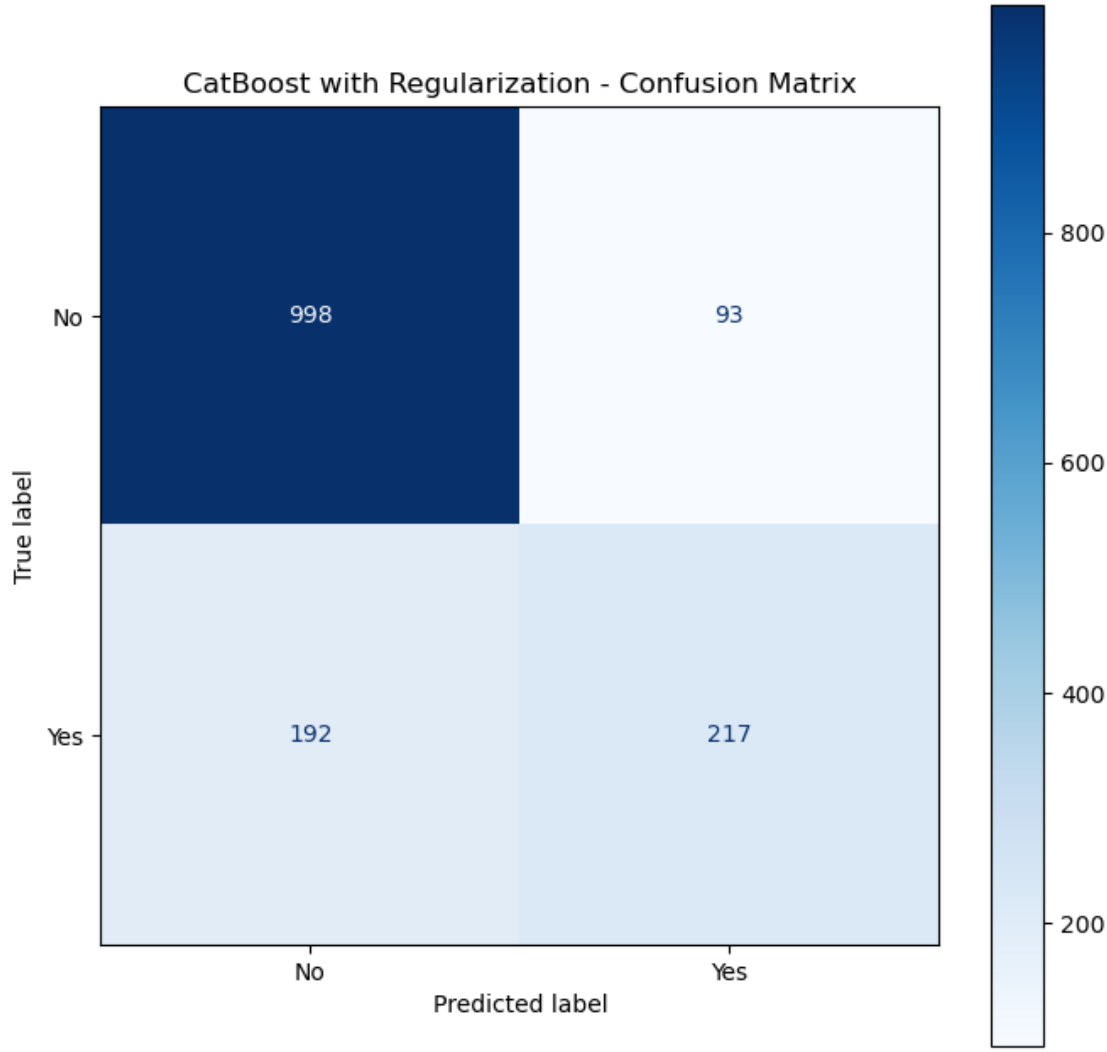
```
    iterations=500,
    learning_rate=0.1,
    depth=6,
    l2_leaf_reg=3,  # L2 regularization to prevent overfitting
    random_seed=42,
    verbose=False
)

# Performing cross-validation
fold = 1
for train_index, test_index in skf.split(X, y):
    X_train_fold, X_test_fold = X.iloc[train_index], X.iloc[test_index]
    y_train_fold, y_test_fold = y.iloc[train_index], y.iloc[test_index]

    # Training the model
    catboost_model.fit(
        X_train_fold,
        y_train_fold,
        cat_features=categorical_features_indices,
        eval_set=(X_test_fold, y_test_fold),
        early_stopping_rounds=50  # Early stopping as additional regularization
    )

    # Making predictions
    y_pred_fold = catboost_model.predict(X_test_fold)

    # Calculating metrics
    fold_accuracy = accuracy_score(y_test_fold, y_pred_fold)
    fold_recall = recall_score(y_test_fold, y_pred_fold, pos_label=1)  # "Yes"␣
↪is positive class (encoded as 1)
    fold_roc_auc = roc_auc_score(y_test_fold, y_pred_fold)
    fold_precision = precision_score(y_test_fold, y_pred_fold, pos_label=1)  #␣
↪"Yes" is positive class (encoded as 1)

    # Storing metrics
    cv_accuracy.append(fold_accuracy)
    cv_recall.append(fold_recall)
    cv_roc_auc.append(fold_roc_auc)
    cv_precision.append(fold_precision)

    print(f"Fold {fold} - Accuracy: {fold_accuracy:.4f}, Recall: {fold_recall:.
↪4f}, "
          f"Precision: {fold_precision:.4f}, ROC-AUC: {fold_roc_auc:.4f}")
    fold += 1

# Calculating mean metrics across all folds
mean_accuracy = np.mean(cv_accuracy)
```

```
mean_recall = np.mean(cv_recall)
mean_roc_auc = np.mean(cv_roc_auc)
mean_precision = np.mean(cv_precision)

print("\nCross-validation results:")
print(f"Mean Accuracy: {mean_accuracy:.4f}")
print(f"Mean Recall: {mean_recall:.4f}")
print(f"Mean ROC-AUC: {mean_roc_auc:.4f}")
print(f"Mean Precision: {mean_precision:.4f}")

# Now let's train on the entire train set and evaluate on test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=42)

# Final model with regularization
final_catboost = CatBoostClassifier(
    iterations=500,
    learning_rate=0.1,
    depth=6,
    l2_leaf_reg=3,  # L2 regularization
    random_seed=42,
    verbose=False
)

# Training on full training set
final_catboost.fit(
    X_train,
    y_train,
    cat_features=categorical_features_indices,
    eval_set=(X_test, y_test),
    early_stopping_rounds=50  # Early stopping
)

# Making predictions on test set
y_pred = final_catboost.predict(X_test)

# Create new lists for final model metrics (THIS IS THE FIX)
accuracy = []
recall = []
roc_auc = []
precision = []

# Calculating final metrics
accuracy.append(round(accuracy_score(y_test, y_pred), 4))
recall.append(round(recall_score(y_test, y_pred, pos_label=1), 4))  # "Yes" is␣
 ↪positive class (encoded as 1)
roc_auc.append(round(roc_auc_score(y_test, y_pred), 4))
```

```
precision.append(round(precision_score(y_test, y_pred, pos_label=1), 4))  #␣
↪"Yes" is positive class (encoded as 1)

model_names = ['Catboost_default']
result_df1 = pd.DataFrame({'Accuracy': accuracy, 'Recall': recall, 'Roc_Auc':␣
↪roc_auc, 'Precision': precision}, index=model_names)
print("\nFinal model performance on test set:")
print(result_df1)

# Display confusion matrix
fig, ax = plt.subplots(figsize=(8, 8))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap=plt.cm.Blues,␣
↪ax=ax,
                                        display_labels=["No", "Yes"] if␣
↪'label_map' in locals() else None)
plt.title('CatBoost with Regularization - Confusion Matrix')
plt.show()

# ===== FIXED STORAGE SECTION =====
# Store the model and results properly
catboost_default = final_catboost  # Stores trained model
default_results = result_df1.copy()  # Stores metrics DataFrame

# Fix column name consistency (Roc_Auc → ROC_AUC)
result_df1 = result_df1.rename(columns={'Roc_Auc': 'ROC_AUC'})  # Rename column
default_results = result_df1.copy()  # Update stored metrics with corrected␣
↪names

# Verification
print("\nStored Default CatBoost Model:")
print(f"- Model object: catboost_default (type: {type(catboost_default)})")
print(f"- Metrics DataFrame:\n{default_results}")
```

```
Fold 1 - Accuracy: 0.8210, Recall: 0.5573, Precision: 0.6986, ROC-AUC: 0.7359
Fold 2 - Accuracy: 0.8100, Recall: 0.5344, Precision: 0.6731, ROC-AUC: 0.7211
Fold 3 - Accuracy: 0.7940, Recall: 0.5399, Precision: 0.6256, ROC-AUC: 0.7123
Fold 4 - Accuracy: 0.8070, Recall: 0.5133, Precision: 0.6750, ROC-AUC: 0.7126
Fold 5 - Accuracy: 0.7980, Recall: 0.4829, Precision: 0.6580, ROC-AUC: 0.6967

Cross-validation results:
Mean Accuracy: 0.8060
Mean Recall: 0.5255
Mean ROC-AUC: 0.7157
Mean Precision: 0.6660

Final model performance on test set:
                  Accuracy  Recall  Roc_Auc  Precision
```

```
Catboost_default      0.81  0.5306   0.7227        0.7
```




```
Stored Default CatBoost Model:
- Model object: catboost_default (type: <class
'catboost.core.CatBoostClassifier'>)
- Metrics DataFrame:
                  Accuracy  Recall  ROC_AUC  Precision
Catboost_default      0.81  0.5306   0.7227        0.7
```

[216]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import LabelEncoder
```

```python
from sklearn.metrics import accuracy_score, recall_score, roc_auc_score,␣
↪precision_score, ConfusionMatrixDisplay
from catboost import CatBoostClassifier
import matplotlib.pyplot as plt

# Limiting to first 5000 rows as specified
# (Assuming X and y are already defined - if they're from a DataFrame, apply␣
↪the limit before this code)
X = X.iloc[:5000]
y = y.iloc[:5000]

# Step 0: Detect categorical columns and prepare cat_features indices
categorical_columns = X.select_dtypes(include=['object']).columns.tolist()

# Convert categorical columns to string dtype to avoid float issues
for col in categorical_columns:
    X[col] = X[col].astype(str)

# Get indices of categorical columns for CatBoost
categorical_features_indices = [X.columns.get_loc(col) for col in␣
↪categorical_columns]

# Encode target variable if it's categorical (Yes/No)
# Check if y contains string labels
if isinstance(y.iloc[0], str):
    print("Converting string labels to numeric...")
    # Create a label encoder
    label_encoder = LabelEncoder()
    # Fit and transform y
    y = pd.Series(label_encoder.fit_transform(y))
    # Store the mapping for reference
    label_map = dict(zip(label_encoder.classes_, label_encoder.
↪transform(label_encoder.classes_)))
    print(f"Label mapping: {label_map}")

# Creating a StratifiedKFold object for cross-validation
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# List to store metrics from each fold
cv_accuracy = []
cv_recall = []
cv_roc_auc = []
cv_precision = []

# Default CatBoost model with regularization
catboost_model = CatBoostClassifier(
    iterations=500,
```

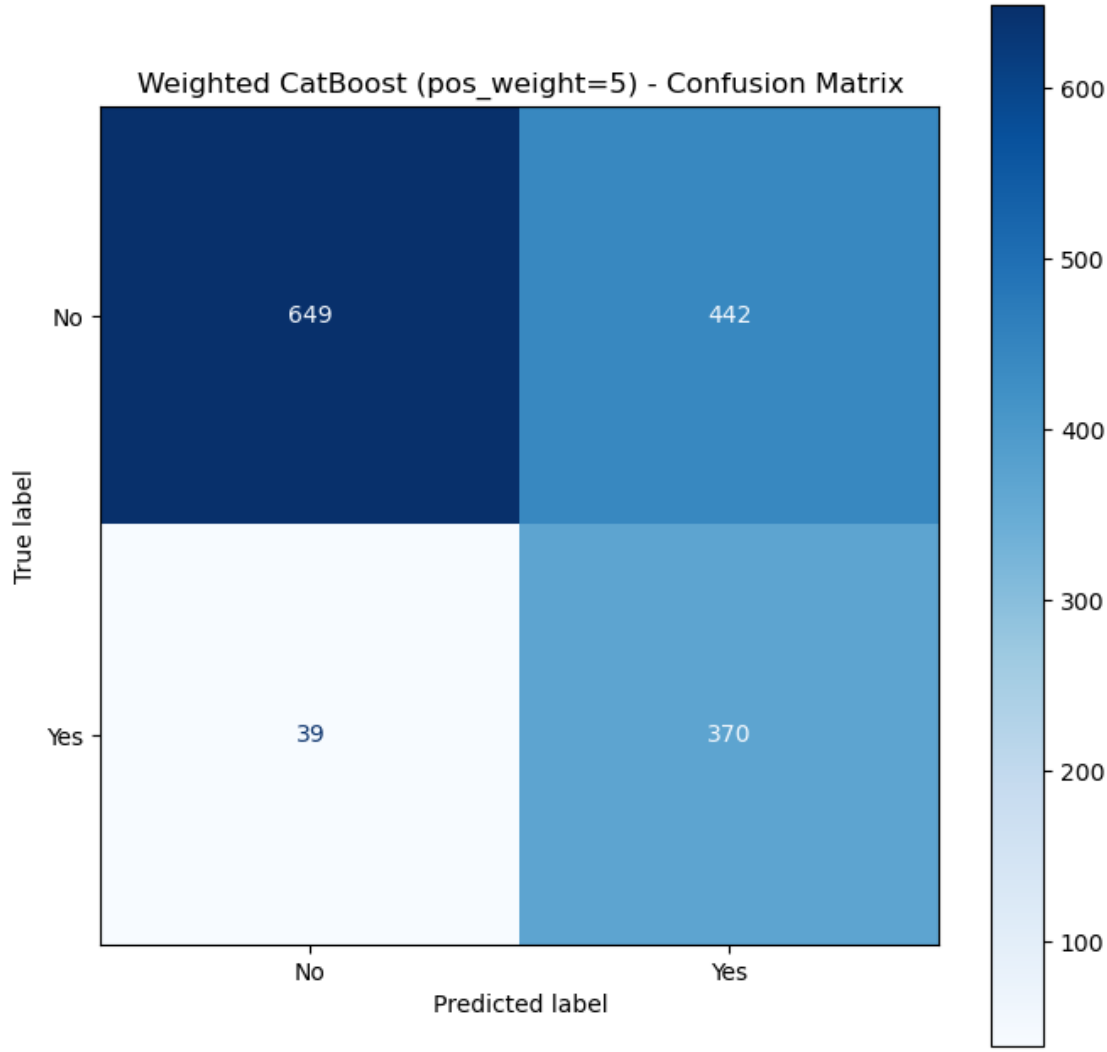
```
    learning_rate=0.1,
    depth=6,
    l2_leaf_reg=3,  # L2 regularization to prevent overfitting
    random_seed=42,
    verbose=False
)

# Performing cross-validation
fold = 1
for train_index, test_index in skf.split(X, y):
    X_train_fold, X_test_fold = X.iloc[train_index], X.iloc[test_index]
    y_train_fold, y_test_fold = y.iloc[train_index], y.iloc[test_index]

    # Training the model
    catboost_model.fit(
        X_train_fold,
        y_train_fold,
        cat_features=categorical_features_indices,   # <-- fixed here
        eval_set=(X_test_fold, y_test_fold),
        early_stopping_rounds=50  # Early stopping as additional regularization
    )

    # Making predictions
    y_pred_fold = catboost_model.predict(X_test_fold)

    # Calculating metrics
    fold_accuracy = accuracy_score(y_test_fold, y_pred_fold)
    fold_recall = recall_score(y_test_fold, y_pred_fold, pos_label=1)  # "Yes"␣
 ↪is positive class (encoded as 1)
    fold_roc_auc = roc_auc_score(y_test_fold, y_pred_fold)
    fold_precision = precision_score(y_test_fold, y_pred_fold, pos_label=1)  #␣
 ↪"Yes" is positive class (encoded as 1)

    # Storing metrics
    cv_accuracy.append(fold_accuracy)
    cv_recall.append(fold_recall)
    cv_roc_auc.append(fold_roc_auc)
    cv_precision.append(fold_precision)

    print(f"Fold {fold} - Accuracy: {fold_accuracy:.4f}, Recall: {fold_recall:.
 ↪4f}, "
          f"Precision: {fold_precision:.4f}, ROC-AUC: {fold_roc_auc:.4f}")
    fold += 1

# Calculating mean metrics across all folds
mean_accuracy = np.mean(cv_accuracy)
mean_recall = np.mean(cv_recall)
```

```
mean_roc_auc = np.mean(cv_roc_auc)
mean_precision = np.mean(cv_precision)

print("\nCross-validation results:")
print(f"Mean Accuracy: {mean_accuracy:.4f}")
print(f"Mean Recall: {mean_recall:.4f}")
print(f"Mean ROC-AUC: {mean_roc_auc:.4f}")
print(f"Mean Precision: {mean_precision:.4f}")

# Now let's train on the entire train set and evaluate on test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=42)

# Final model with regularization
final_catboost = CatBoostClassifier(
    iterations=500,
    learning_rate=0.1,
    depth=6,
    l2_leaf_reg=3,  # L2 regularization
    scale_pos_weight=5,  # <-- just this line added
    random_seed=42,
    verbose=False
)

# Training on full training set
final_catboost.fit(
    X_train,
    y_train,
    cat_features=categorical_features_indices,   # <-- fixed here too
    eval_set=(X_test, y_test),
    early_stopping_rounds=50  # Early stopping
)

# Making predictions on test set
y_pred = final_catboost.predict(X_test)

# Create new lists for final model metrics (THIS IS THE FIX)
accuracy = []
recall = []
roc_auc = []
precision = []

# Calculating final metrics
accuracy.append(round(accuracy_score(y_test, y_pred), 4))
recall.append(round(recall_score(y_test, y_pred, pos_label=1), 4))  # "Yes" is␣
 ↪positive class (encoded as 1)
roc_auc.append(round(roc_auc_score(y_test, y_pred), 4))
```

```
precision.append(round(precision_score(y_test, y_pred, pos_label=1), 4))  #␣
↪"Yes" is positive class (encoded as 1)

model_names = ['Catboost_with_adjusted_weight_5']
result_df2 = pd.DataFrame({'Accuracy': accuracy, 'Recall': recall, 'Roc_Auc':␣
↪roc_auc, 'Precision': precision}, index=model_names)
print("\nFinal model performance on test set:")
print(result_df2)

# Display confusion matrix
fig, ax = plt.subplots(figsize=(8, 8))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
                                        cmap=plt.cm.Blues,
                                        ax=ax,
                                        display_labels=["No", "Yes"] if␣
↪'label_map' in locals() else None)
plt.title('Weighted CatBoost (pos_weight=5) - Confusion Matrix')
plt.show()

# ===== PROPER STORAGE =====
# 1. Store the trained model
catboost_5 = final_catboost

# 2. Standardize column names and store metrics
result_df2 = result_df2.rename(columns={'Roc_Auc': 'ROC_AUC'})  # Fix column␣
↪name
weighted_results = result_df2.copy()  # Store metrics with consistent naming

# Verification
print("\nStored Weighted CatBoost Model:")
print(f"- Model object: catboost_5 (type: {type(catboost_5)})")
print(f"- Metrics:\n{weighted_results}")
```

```
Fold 1 - Accuracy: 0.8160, Recall: 0.5344, Precision: 0.6931, ROC-AUC: 0.7252
Fold 2 - Accuracy: 0.8100, Recall: 0.5382, Precision: 0.6714, ROC-AUC: 0.7223
Fold 3 - Accuracy: 0.7910, Recall: 0.5209, Precision: 0.6227, ROC-AUC: 0.7041
Fold 4 - Accuracy: 0.8090, Recall: 0.4867, Precision: 0.6957, ROC-AUC: 0.7054
Fold 5 - Accuracy: 0.8080, Recall: 0.4905, Precision: 0.6898, ROC-AUC: 0.7059

Cross-validation results:
Mean Accuracy: 0.8068
Mean Recall: 0.5141
Mean ROC-AUC: 0.7126
Mean Precision: 0.6745

Final model performance on test set:
                                Accuracy  Recall  Roc_Auc  Precision
```

```
Catboost_with_adjusted_weight_5    0.6793  0.9046   0.7498     0.4557
```




```
Stored Weighted CatBoost Model:
- Model object: catboost_5 (type: <class 'catboost.core.CatBoostClassifier'>)
- Metrics:
                                Accuracy  Recall  ROC_AUC  Precision
Catboost_with_adjusted_weight_5    0.6793  0.9046   0.7498     0.4557
```

[218]:
```python
# this code got quite  alot of error so we will redo the entire data processing

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import LabelEncoder
```

```python
from sklearn.metrics import accuracy_score, recall_score, roc_auc_score,␣
↪precision_score, ConfusionMatrixDisplay
import lightgbm as lgb
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Load the dataset
print("Loading dataset...")
df = pd.read_csv('WA_Fn-UseC_-Telco-Customer-Churn.csv')
df= df = df.iloc[:5000]
# Display basic information about the dataset
print(f"Original dataset shape: {df.shape}")
print("\nFirst few rows of the dataset:")
print(df.head())

# Limit to first 5000 rows as required
df1 = df.iloc[:5000].copy()
print(f"\nReduced dataset shape: {df1.shape}")

# Check for missing values
print("\nChecking for missing values:")
print(df1.isnull().sum())

# Data preprocessing
print("\nPerforming data preprocessing...")

# Handle TotalCharges column - convert blank to NaN and then to numeric
df1['TotalCharges'] = pd.to_numeric(df1['TotalCharges'], errors='coerce')
print(f"Number of NaN values in TotalCharges after conversion:␣
↪{df1['TotalCharges'].isna().sum()}")

# Drop CustomerID as it's not useful for prediction
if 'customerID' in df1.columns:
    df1 = df1.drop('customerID', axis=1)
    print("Dropped customerID column")

# Replace 'No internet service' with 'No' in relevant columns
cols_to_replace = ['OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
                   'TechSupport', 'StreamingTV', 'StreamingMovies']
df1[cols_to_replace] = df1[cols_to_replace].replace('No internet service', 'No')
print("Replaced 'No internet service' with 'No' in specified columns")

# Handle missing values - fill with mean or drop rows
if df1['TotalCharges'].isna().sum() > 0:
    # Option 1: Fill with mean
    df1['TotalCharges'] = df1['TotalCharges'].fillna(df1['TotalCharges'].mean())
```

```
    print("Filled NaN values in TotalCharges with mean")

    # Option 2: Drop rows with NaN (uncomment if preferred)
    # df1 = df1.dropna()
    # print(f"Dropped rows with NaN values, new shape: {df1.shape}")

# Encode categorical features
print("\nEncoding categorical features...")
categorical_columns = df1.select_dtypes(include=['object']).columns.tolist()
print(f"Categorical columns to encode: {categorical_columns}")

# Special handling for target variable
target_column = 'Churn'
label_encoder = LabelEncoder()
df1[target_column] = label_encoder.fit_transform(df1[target_column])
label_map = dict(zip(label_encoder.classes_, label_encoder.
 ↪transform(label_encoder.classes_)))
print(f"Target variable encoding: {label_map}")

# Remove target from categorical columns list
if target_column in categorical_columns:
    categorical_columns.remove(target_column)

# Encode remaining categorical features
encoders = {}
for col in categorical_columns:
    le = LabelEncoder()
    df1[col] = le.fit_transform(df1[col])
    encoders[col] = le
    print(f"Encoded {col}: {dict(zip(le.classes_, le.transform(le.classes_)))}")

# Prepare features and target
X = df1.drop(target_column, axis=1)
y = df1[target_column]
print(f"\nFeatures shape: {X.shape}, Target shape: {y.shape}")

# Identify categorical feature indices for LightGBM
categorical_features_indices = [X.columns.get_loc(col) for col in␣
 ↪categorical_columns if col in X.columns]
print(f"Categorical feature indices: {categorical_features_indices}")

# Calculate class weights for imbalanced data
class_counts = y.value_counts()
scale_pos_weight = class_counts[0] / class_counts[1] if 1 in class_counts.index␣
 ↪else 1.0
print(f"\nClass distribution: {class_counts}")
print(f"Using scale_pos_weight: {scale_pos_weight:.2f}")
```

```python
# Creating a StratifiedKFold object for cross-validation
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# List to store metrics from each fold
cv_accuracy = []
cv_recall = []
cv_roc_auc = []
cv_precision = []

# LightGBM parameters - adapted for smaller dataset (5000 rows)
lgb_params = {
    'objective': 'binary',
    'metric': 'binary_logloss',
    'boosting_type': 'gbdt',
    'learning_rate': 0.05,        # Lower learning rate for smaller dataset
    'n_estimators': 300,          # Fewer trees for smaller dataset
    'max_depth': 4,               # Reduced depth to prevent overfitting
    'num_leaves': 15,             # Fewer leaves than default for small data
    'min_child_samples': 20,      # Require more samples per leaf for stability
    'reg_alpha': 0.1,             # L1 regularization
    'reg_lambda': 0.3,            # L2 regularization
    'colsample_bytree': 0.8,      # Use 80% of features per tree
    'subsample': 0.8,             # Use 80% of samples per tree
    'scale_pos_weight': 5,  # Add weight to handle class imbalance
    'random_state': 42,
    'verbose': -1
}

# Function to create LightGBM datasets
def create_lgb_dataset(X, y, categorical_features=None):
    return lgb.Dataset(X, label=y, categorical_feature=categorical_features)

# Performing cross-validation
print("\nPerforming 5-fold cross-validation...")
fold = 1
for train_index, test_index in skf.split(X, y):
    X_train_fold, X_test_fold = X.iloc[train_index], X.iloc[test_index]
    y_train_fold, y_test_fold = y.iloc[train_index], y.iloc[test_index]

    # Create LightGBM datasets
    train_data = create_lgb_dataset(X_train_fold, y_train_fold,␣
 ↪categorical_features_indices)
    valid_data = create_lgb_dataset(X_test_fold, y_test_fold,␣
 ↪categorical_features_indices)

    # Training the model
```

```
    lgb_model = lgb.train(
        params=lgb_params,
        train_set=train_data,
        valid_sets=[valid_data],
        callbacks=[lgb.early_stopping(stopping_rounds=30)]
    )

    # Making predictions
    y_pred_proba = lgb_model.predict(X_test_fold, num_iteration=lgb_model.
 ↪best_iteration)
    y_pred_fold = (y_pred_proba > 0.5).astype(int)

    # Calculating metrics
    fold_accuracy = accuracy_score(y_test_fold, y_pred_fold)
    fold_recall = recall_score(y_test_fold, y_pred_fold, pos_label=1)  # "Yes"␣
 ↪is positive class (encoded as 1)
    fold_roc_auc = roc_auc_score(y_test_fold, y_pred_proba)
    fold_precision = precision_score(y_test_fold, y_pred_fold, pos_label=1)

    # Storing metrics
    cv_accuracy.append(fold_accuracy)
    cv_recall.append(fold_recall)
    cv_roc_auc.append(fold_roc_auc)
    cv_precision.append(fold_precision)

    print(f"Fold {fold} - Accuracy: {fold_accuracy:.4f}, Recall: {fold_recall:.
 ↪4f}, "
          f"Precision: {fold_precision:.4f}, ROC-AUC: {fold_roc_auc:.4f}")
    fold += 1

# Calculating mean metrics across all folds
mean_accuracy = np.mean(cv_accuracy)
mean_recall = np.mean(cv_recall)
mean_roc_auc = np.mean(cv_roc_auc)
mean_precision = np.mean(cv_precision)

print("\nCross-validation results:")
print(f"Mean Accuracy: {mean_accuracy:.4f}")
print(f"Mean Recall: {mean_recall:.4f}")
print(f"Mean ROC-AUC: {mean_roc_auc:.4f}")
print(f"Mean Precision: {mean_precision:.4f}")

# Now let's train on the entire train set and evaluate on test set
print("\nTraining final model on 70% train, 30% test split...")
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=42, stratify=y)
```

```python
# Create LightGBM datasets
train_data = create_lgb_dataset(X_train, y_train, categorical_features_indices)
valid_data = create_lgb_dataset(X_test, y_test, categorical_features_indices)

# Final model training
final_lgb = lgb.train(
    params=lgb_params,
    train_set=train_data,
    valid_sets=[valid_data],
    callbacks=[lgb.early_stopping(stopping_rounds=30)]
)

# Making predictions on test set
y_pred_proba = final_lgb.predict(X_test, num_iteration=final_lgb.best_iteration)
y_pred = (y_pred_proba > 0.5).astype(int)

# Create new lists for final model metrics
accuracy = []
recall = []
roc_auc = []
precision = []

# Calculating final metrics
accuracy.append(round(accuracy_score(y_test, y_pred), 4))
recall.append(round(recall_score(y_test, y_pred, pos_label=1), 4))
roc_auc.append(round(roc_auc_score(y_test, y_pred_proba), 4))
precision.append(round(precision_score(y_test, y_pred, pos_label=1), 4))

model_names = ['LightGBM']
result_df1 = pd.DataFrame({'Accuracy': accuracy, 'Recall': recall, 'ROC_AUC':␣
 ↪roc_auc, 'Precision': precision}, index=model_names)
print("\nFinal model performance on test set:")
print(result_df1)

# Display confusion matrix
print("\nGenerating confusion matrix...")
fig, ax = plt.subplots(figsize=(8, 8))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
                                        cmap=plt.cm.Blues,
                                        ax=ax,
                                        display_labels=["No", "Yes"])
plt.title('LightGBM - Confusion Matrix')
plt.savefig('lightgbm_confusion_matrix.png')  # Unique filename
plt.show()

# ===== PROPER STORAGE =====
# 1. Store the trained model
```

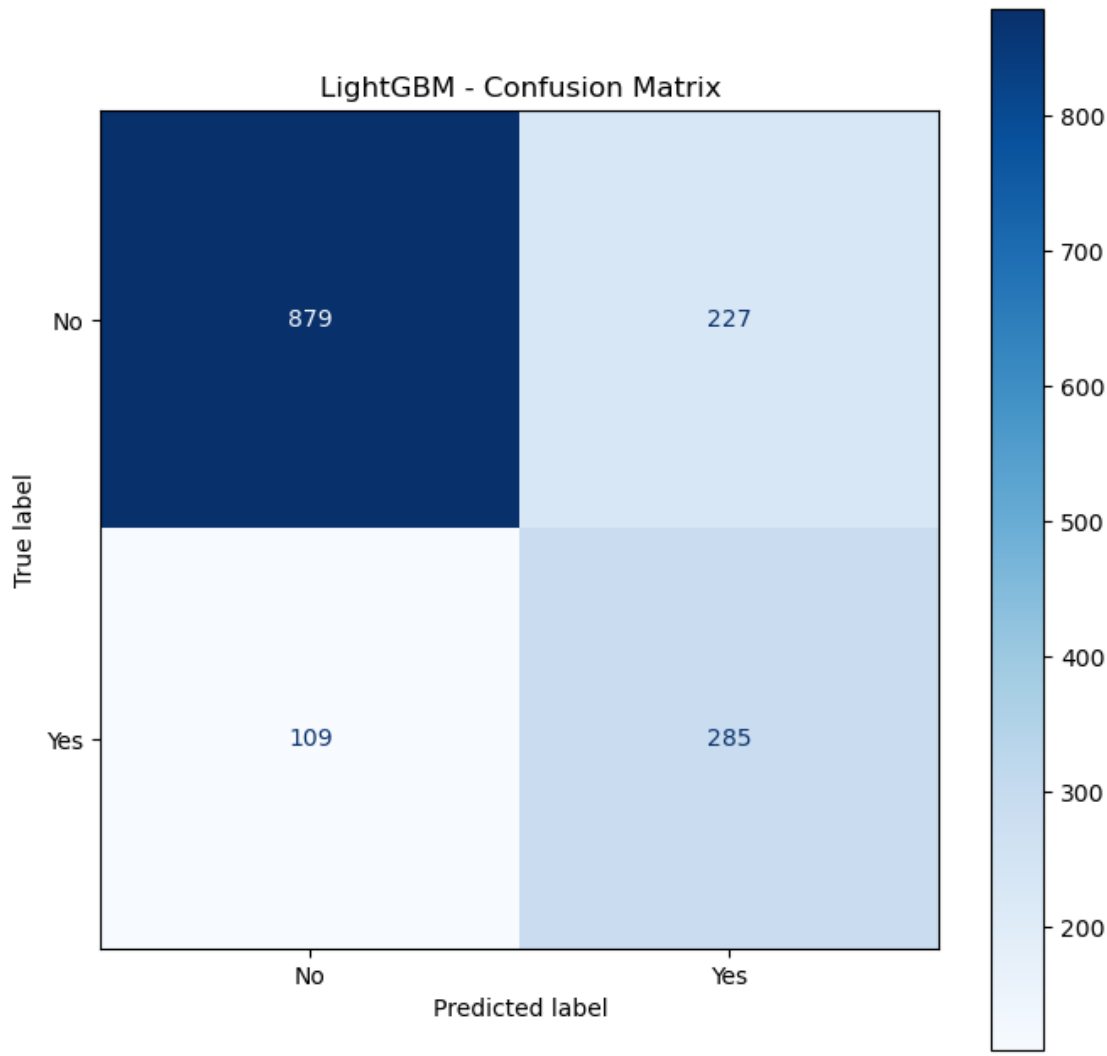
```python
lightgbm_model = final_lgb

# 2. Store metrics with standardized column names
result_df3 = result_df1.rename(columns={
    'ROC_AUC': 'ROC_AUC',  # Ensures consistency (even if already correct)
    'Roc_Auc': 'ROC_AUC'   # Catches any lowercase variants
})

# 3. Verification
print("\n=== LightGBM Storage Verification ===")
print(f"Model stored as: lightgbm_model (type: {type(lightgbm_model)})")
print("Metrics DataFrame columns:", result_df3.columns.tolist())
print("\nFinal Metrics:")
print(result_df3)
```

```
Loading dataset…
Original dataset shape: (5000, 21)

First few rows of the dataset:
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  … DeviceProtection  \
0  No phone service             DSL             No  …               No
1                No             DSL            Yes  …              Yes
2                No             DSL            Yes  …               No
3  No phone service             DSL            Yes  …              Yes
4                No     Fiber optic             No  …               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
```

```
Reduced dataset shape: (5000, 21)

Checking for missing values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Performing data preprocessing…
Number of NaN values in TotalCharges after conversion: 8
Dropped customerID column
Replaced 'No internet service' with 'No' in specified columns
Filled NaN values in TotalCharges with mean

Encoding categorical features…
Categorical columns to encode: ['gender', 'Partner', 'Dependents',
'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod', 'Churn']
Target variable encoding: {'No': 0, 'Yes': 1}
Encoded gender: {'Female': 0, 'Male': 1}
Encoded Partner: {'No': 0, 'Yes': 1}
Encoded Dependents: {'No': 0, 'Yes': 1}
Encoded PhoneService: {'No': 0, 'Yes': 1}
Encoded MultipleLines: {'No': 0, 'No phone service': 1, 'Yes': 2}
Encoded InternetService: {'DSL': 0, 'Fiber optic': 1, 'No': 2}
Encoded OnlineSecurity: {'No': 0, 'Yes': 1}
Encoded OnlineBackup: {'No': 0, 'Yes': 1}
Encoded DeviceProtection: {'No': 0, 'Yes': 1}
```

```
Encoded TechSupport: {'No': 0, 'Yes': 1}
Encoded StreamingTV: {'No': 0, 'Yes': 1}
Encoded StreamingMovies: {'No': 0, 'Yes': 1}
Encoded Contract: {'Month-to-month': 0, 'One year': 1, 'Two year': 2}
Encoded PaperlessBilling: {'No': 0, 'Yes': 1}
Encoded PaymentMethod: {'Bank transfer (automatic)': 0, 'Credit card
(automatic)': 1, 'Electronic check': 2, 'Mailed check': 3}

Features shape: (5000, 19), Target shape: (5000,)
Categorical feature indices: [0, 2, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
16]

Class distribution: Churn
0    3687
1    1313
Name: count, dtype: int64
Using scale_pos_weight: 2.81

Performing 5-fold cross-validation…
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[16]    valid_0's binary_logloss: 0.492204
Fold 1 - Accuracy: 0.7740, Recall: 0.7481, Precision: 0.5506, ROC-AUC: 0.8594
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[14]    valid_0's binary_logloss: 0.499279
Fold 2 - Accuracy: 0.7710, Recall: 0.6794, Precision: 0.5511, ROC-AUC: 0.8462
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[11]    valid_0's binary_logloss: 0.514244
Fold 3 - Accuracy: 0.7800, Recall: 0.5475, Precision: 0.5878, ROC-AUC: 0.8219
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[14]    valid_0's binary_logloss: 0.496929
Fold 4 - Accuracy: 0.7960, Recall: 0.7224, Precision: 0.5919, ROC-AUC: 0.8485
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[13]    valid_0's binary_logloss: 0.502587
Fold 5 - Accuracy: 0.7850, Recall: 0.6540, Precision: 0.5811, ROC-AUC: 0.8443

Cross-validation results:
Mean Accuracy: 0.7812
Mean Recall: 0.6703
Mean ROC-AUC: 0.8440
Mean Precision: 0.5725

Training final model on 70% train, 30% test split…
Training until validation scores don't improve for 30 rounds
```

```
Early stopping, best iteration is:
[14]    valid_0's binary_logloss: 0.502392

Final model performance on test set:
          Accuracy  Recall  ROC_AUC  Precision
LightGBM     0.776  0.7234   0.8392     0.5566

Generating confusion matrix…
```




```
=== LightGBM Storage Verification ===
Model stored as: lightgbm_model (type: <class 'lightgbm.basic.Booster'>)
Metrics DataFrame columns: ['Accuracy', 'Recall', 'ROC_AUC', 'Precision']

Final Metrics:
```

```
          Accuracy  Recall  ROC_AUC  Precision
LightGBM     0.776  0.7234   0.8392     0.5566
```

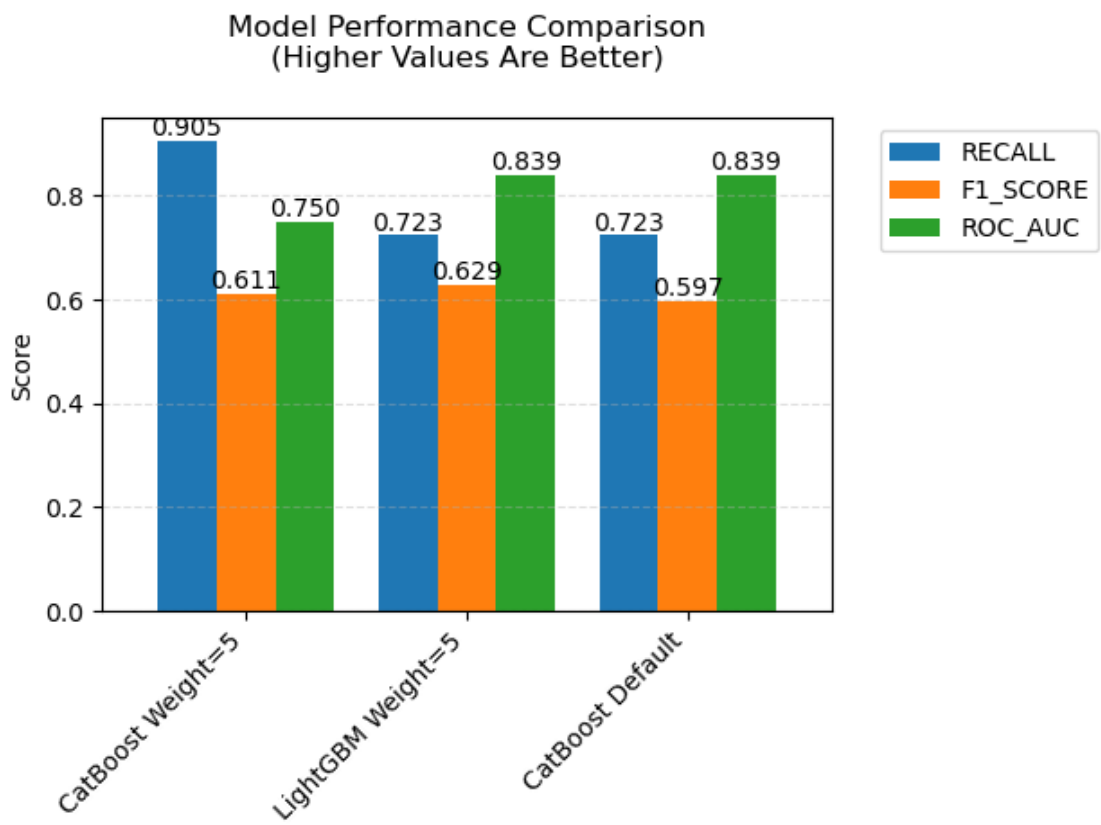
[250]:
```python
from sklearn.metrics import f1_score
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# 1. Verify and standardize input DataFrames
print("=== Input Verification ===")
print("CatBoost Default columns:", result_df1.columns.tolist())
print("CatBoost Weighted columns:", result_df2.columns.tolist())
print("LightGBM columns:", result_df3.columns.tolist())

# Standardize column names for each DataFrame
for df in [result_df1, result_df2, result_df3]:
    df.columns = df.columns.str.upper()
    if 'ROC_AUC' not in df.columns and 'ROC_AUC' in df.columns.str.upper():
        df = df.rename(columns={'ROC_AUC': 'ROC_AUC'})

# 2. Create properly labeled comparison DataFrame - using CLEAR model names
comparison = pd.concat([
    result_df1.rename(index={result_df1.index[0]: 'CatBoost Default'}),
    result_df2.rename(index={result_df2.index[0]: 'CatBoost Weight=5'}),
    result_df3.rename(index={result_df3.index[0]: 'LightGBM Weight=5'})
], axis=0)

# 3. Calculate F1-scores with proper prediction handling
models = {
    'CatBoost Default': catboost_default,
    'CatBoost Weight=5': catboost_5,
    'LightGBM Weight=5': lightgbm_model
}

f1_scores = []
for name, model in models.items():
    try:
        # First try direct prediction (works for CatBoost)
        y_pred = model.predict(X_test)
        if y_pred.dtype == float:  # If probabilities come out
            y_pred = (y_pred > 0.5).astype(int)
    except AttributeError:
        # LightGBM specific handling
        y_pred = (model.predict(X_test) > 0.5).astype(int)
    f1_scores.append(f1_score(y_test, y_pred, pos_label=1))

comparison['F1_SCORE'] = f1_scores
```

```python
# 4. Sort models by importance for churn detection
comparison_sorted = comparison.sort_values(
    by=['RECALL', 'F1_SCORE', 'ROC_AUC'],
    ascending=[False, False, False]
)

# 5. Visual comparison with DYNAMICALLY assigned labels
plt.figure(figsize=(12, 6))
metrics = ['RECALL', 'F1_SCORE', 'ROC_AUC']

# Use the index of the sorted dataframe to get the correct model names
ax = comparison_sorted[metrics].plot(
    kind='bar',
    color=['#1f77b4', '#ff7f0e', '#2ca02c'],
    width=0.8
)

# Use the actual index names from the sorted dataframe
ax.set_xticklabels(
    comparison_sorted.index.tolist(),  # This will use correct sorted order
    rotation=45,
    ha='right'
)

plt.title("Model Performance Comparison\n(Higher Values Are Better)", pad=20)
plt.ylabel("Score")
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()

# Add value labels on bars
for p in ax.patches:
    ax.annotate(
        f"{p.get_height():.3f}",
        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center',
        va='center',
        xytext=(0, 5),
        textcoords='offset points'
    )

plt.show()

# 6. Final ranked output with highlighting
print("\n=== Final Model Ranking ===")
def highlight_max(s):
```

```
    is_max = s == s.max()
    return ['background-color: lightgreen' if v else '' for v in is_max]

display(comparison_sorted.style
        .apply(highlight_max, subset=['RECALL', 'F1_SCORE', 'ROC_AUC'])
        .format("{:.4f}"))

# 7. Print an explicit verification of the model performance to double-check
print("\n=== Model Performance Verification ===")
for model_name in comparison_sorted.index:
    print(f"{model_name}:")
    for metric in metrics:
        print(f"  {metric}: {comparison_sorted.loc[model_name, metric]:.4f}")
```

```
=== Input Verification ===
CatBoost Default columns: ['ACCURACY', 'RECALL', 'ROC_AUC', 'PRECISION']
CatBoost Weighted columns: ['ACCURACY', 'RECALL', 'ROC_AUC', 'PRECISION']
LightGBM columns: ['ACCURACY', 'RECALL', 'ROC_AUC', 'PRECISION']
```

```
<Figure size 1200x600 with 0 Axes>
```



```
=== Final Model Ranking ===
```

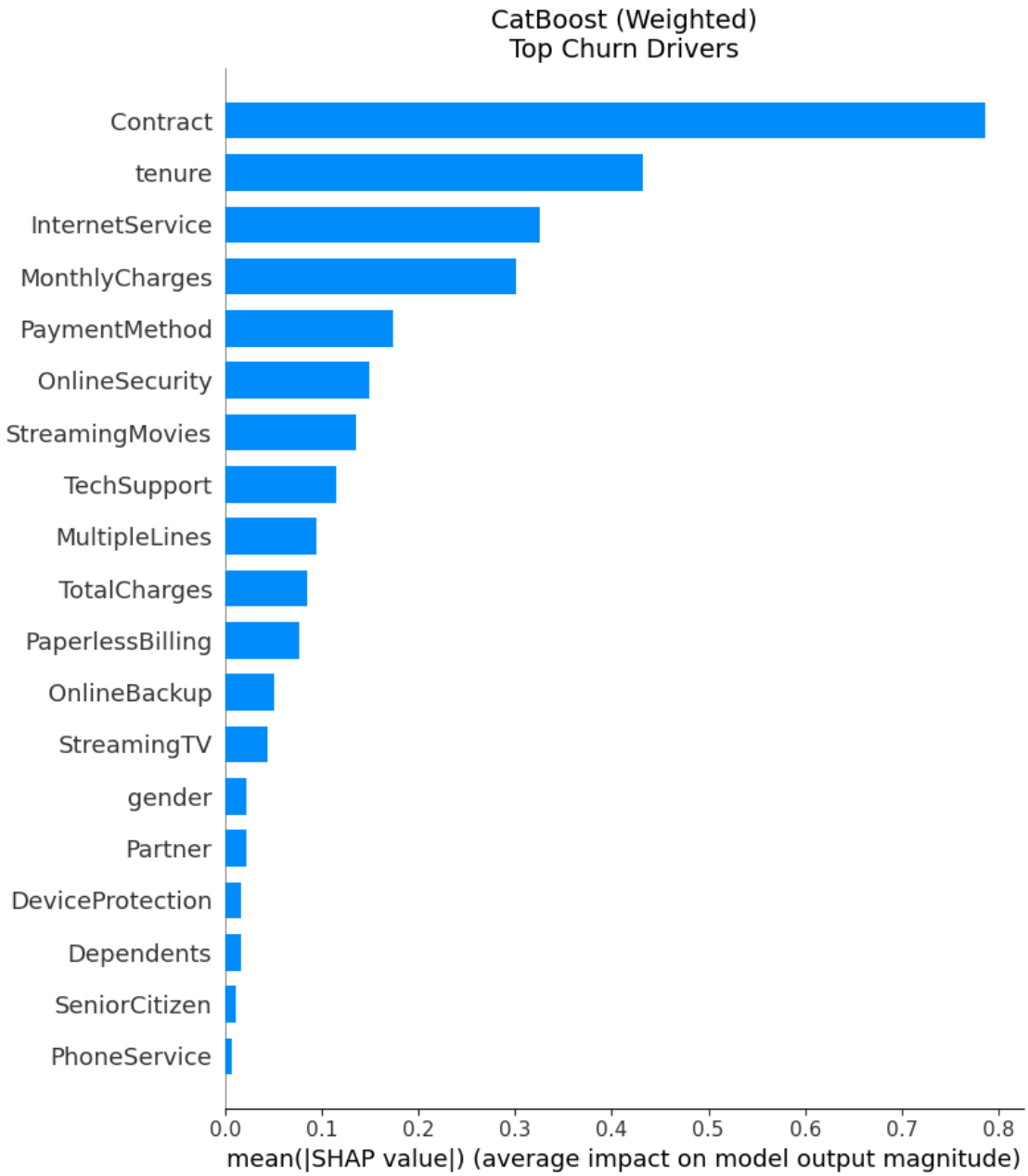
```
<pandas.io.formats.style.Styler at 0x1c7a73d4260>
```

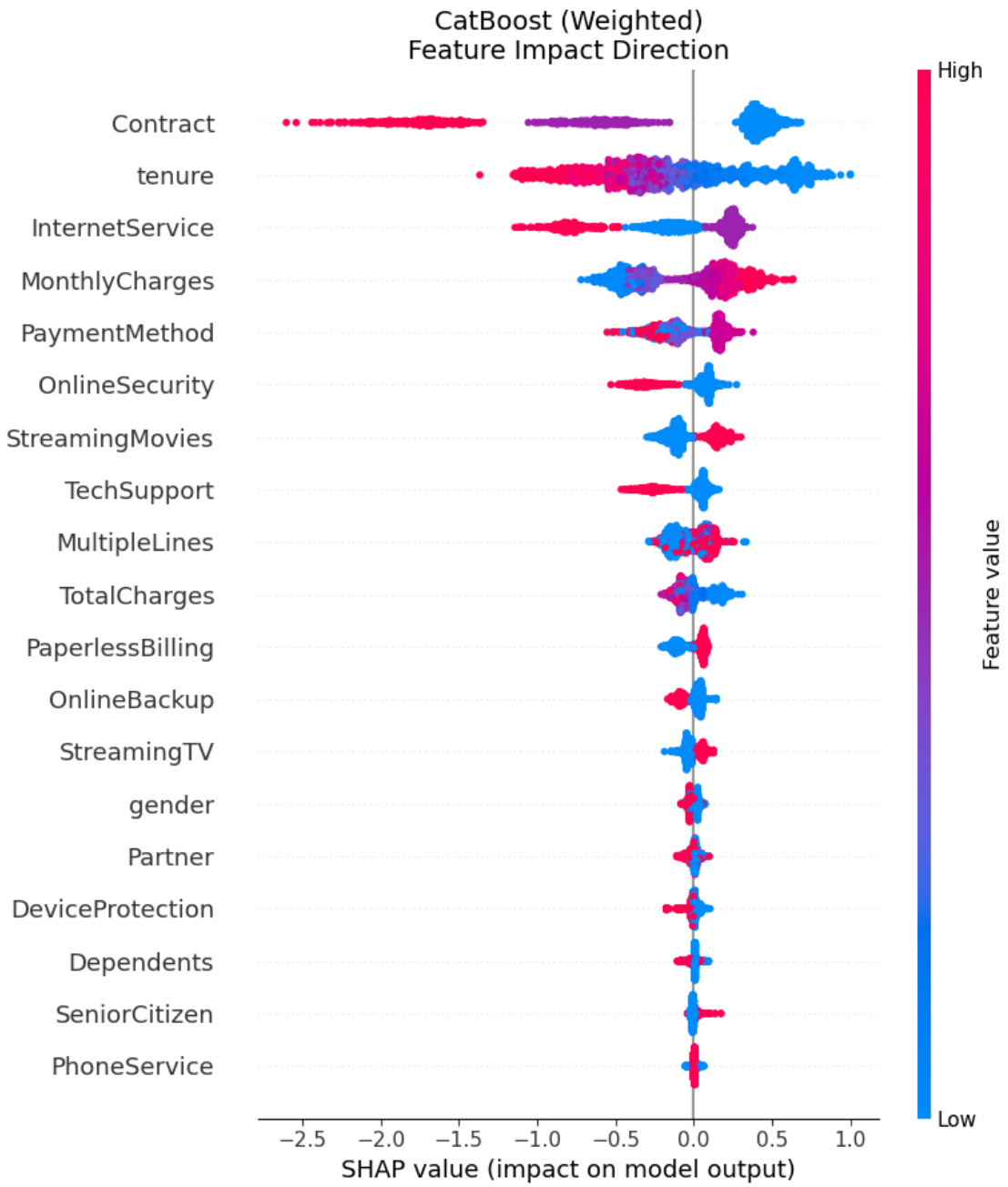
```
=== Model Performance Verification ===
CatBoost Weight=5:
  RECALL: 0.9046
  F1_SCORE: 0.6107
  ROC_AUC: 0.7498
LightGBM Weight=5:
  RECALL: 0.7234
  F1_SCORE: 0.6291
  ROC_AUC: 0.8392
CatBoost Default:
  RECALL: 0.7234
  F1_SCORE: 0.5965
  ROC_AUC: 0.8392
```

[245]:
```python
import shap
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Initialize SHAP explainers
cb_explainer = shap.TreeExplainer(catboost_5)
lgbm_explainer = shap.TreeExplainer(lightgbm_model)

def generate_shap_insights(model, explainer, X_test, model_name):
    """Generate SHAP insights with Telco-specific interpretations"""
    print(f"\n {model_name} Churn Drivers Analysis")

    # Get SHAP values
    shap_values = explainer.shap_values(X_test)

    # Handle binary classification output
    if isinstance(shap_values, list):
        shap_values = shap_values[1]  # Take values for churn class (1)

    # 1. Feature Importance Plot
    plt.figure(figsize=(10,6))
    shap.summary_plot(shap_values, X_test, plot_type="bar", show=False)
    plt.title(f"{model_name}\nTop Churn Drivers", fontsize=14)
    plt.tight_layout()
    plt.show()

    # 2. Detailed Impact Plot
    plt.figure(figsize=(10,6))
```

```
    shap.summary_plot(shap_values, X_test, show=False)
    plt.title(f"{model_name}\nFeature Impact Direction", fontsize=14)
    plt.tight_layout()
    plt.show()

    # 3. Telco-specific insights
    top_features = pd.DataFrame({
        'Feature': X_test.columns,
        'Impact_Score': np.abs(shap_values).mean(axis=0)
    }).sort_values('Impact_Score', ascending=False).head(5)

    print("\n  Top 5 Churn Drivers:")
    display(top_features)

    # 4. High-risk customer identification
    risk_scores = shap_values.sum(axis=1)
    high_risk = X_test.assign(Risk_Score=risk_scores).nlargest(5, 'Risk_Score')

    print("\n  Highest-Risk Customers:")
    display(high_risk[['Contract', 'MonthlyCharges', 'InternetService',␣
 ↪'Risk_Score']])

# Generate insights for both models
generate_shap_insights(catboost_5, cb_explainer, X_test, "CatBoost (Weighted)")
generate_shap_insights(lightgbm_model, lgbm_explainer, X_test, "LightGBM␣
 ↪(Weighted)")
```

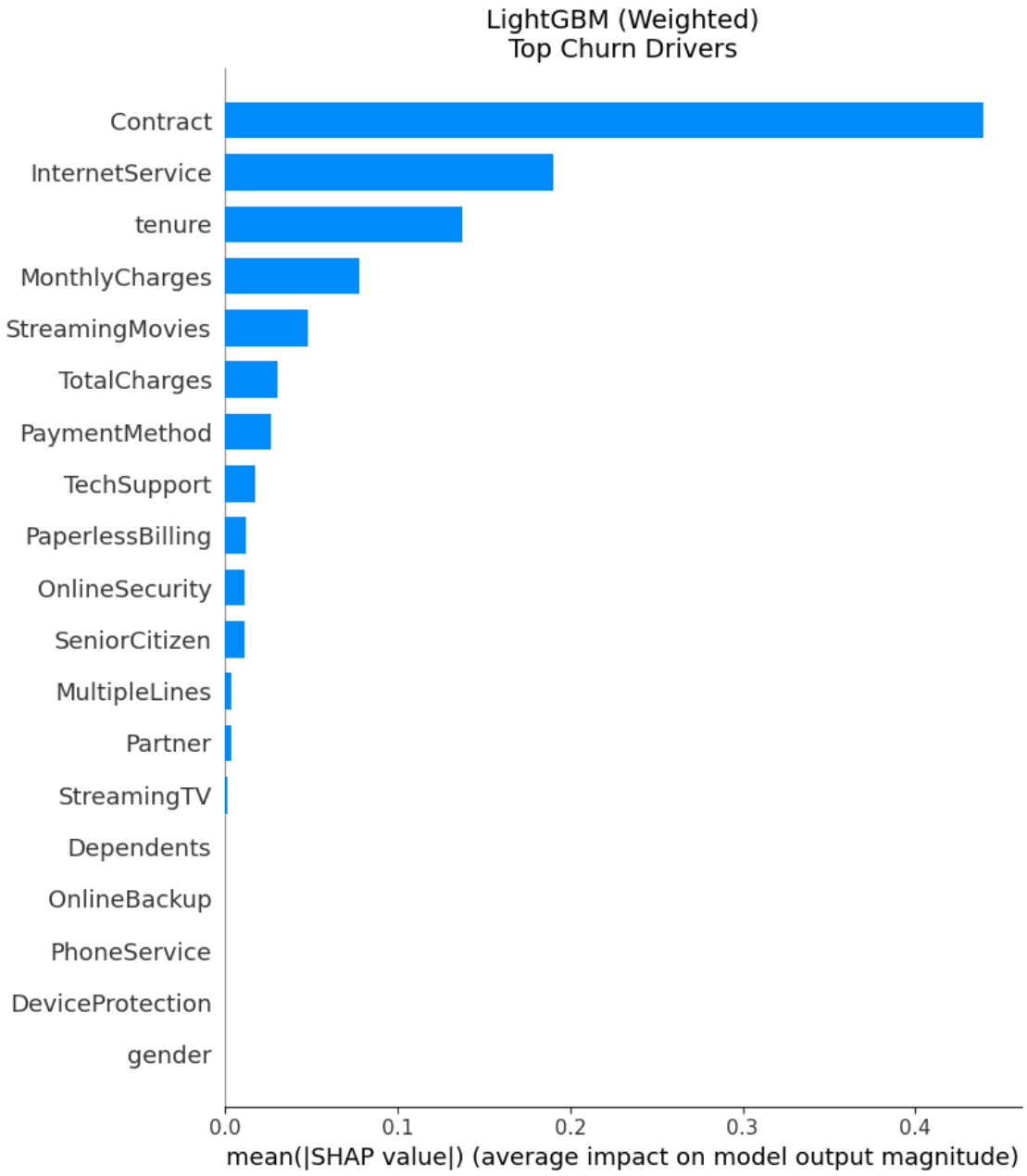
```
CatBoost (Weighted) Churn Drivers Analysis
```

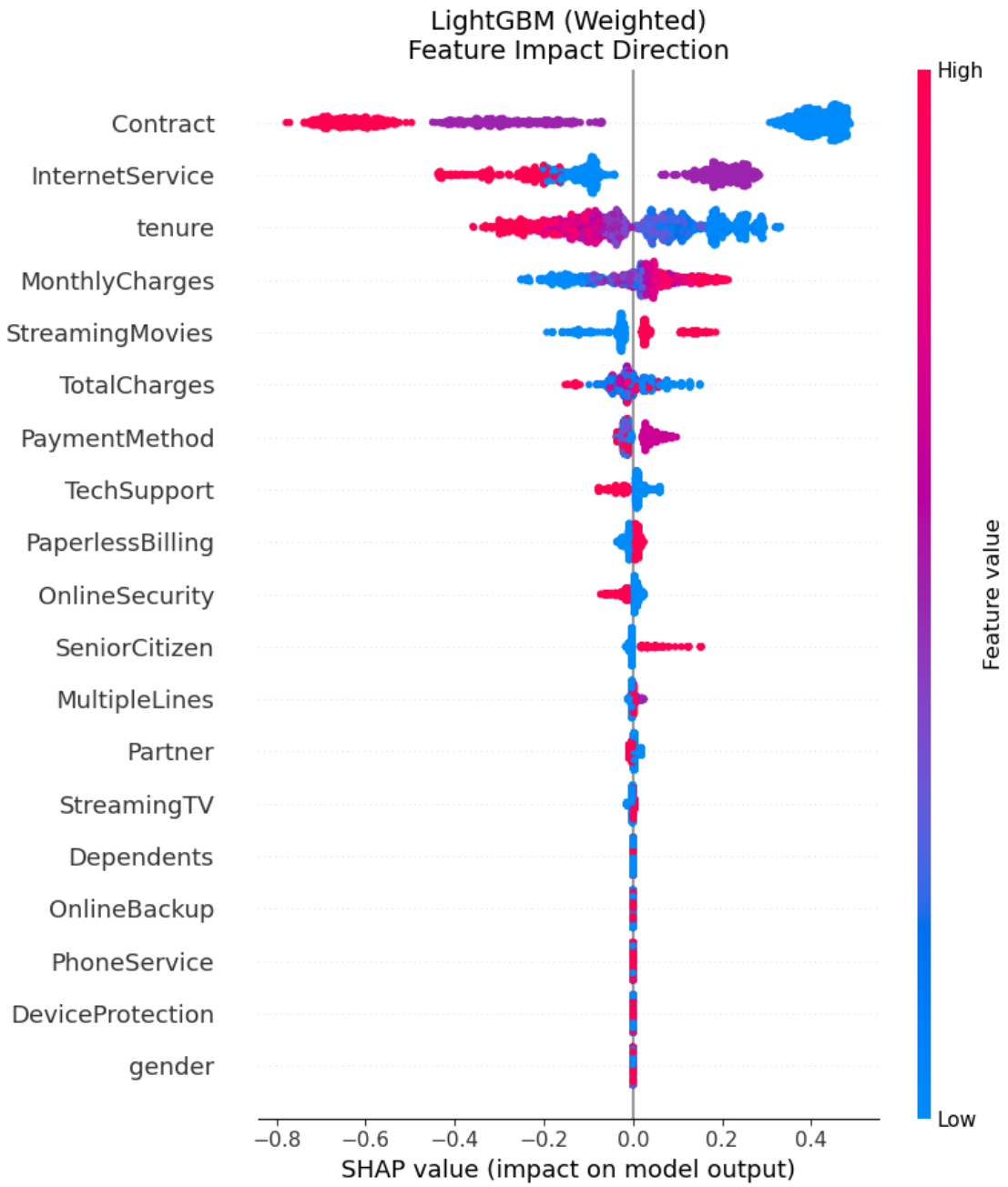

CatBoost (Weighted)
Top Churn Drivers
Contract
tenure
InternetService
MonthlyCharges
PaymentMethod
OnlineSecurity
StreamingMovies
TechSupport
MultipleLines
TotalCharges
PaperlessBilling
OnlineBackup
StreamingTV
gender
Partner
DeviceProtection
Dependents
SeniorCitizen
PhoneService
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
mean(|SHAP value|) (average impact on model output magnitude)




```
Top 5 Churn Drivers:

            Feature  Impact_Score
14         Contract      0.786677
4            tenure      0.432659
7   InternetService      0.325700
17   MonthlyCharges      0.301285
```

```
16    PaymentMethod      0.174314


 Highest-Risk Customers:

      Contract  MonthlyCharges  InternetService  Risk_Score
1704         0           99.75                1    3.282942
1976         0           93.55                1    2.871176
4459         0          100.25                1    2.809875
534          0           85.00                1    2.610439
1600         0           88.80                1    2.557038


 LightGBM (Weighted) Churn Drivers Analysis
```


LightGBM (Weighted)
Top Churn Drivers
Contract
InternetService
tenure
MonthlyCharges
StreamingMovies
TotalCharges
PaymentMethod
TechSupport
PaperlessBilling
OnlineSecurity
SeniorCitizen
MultipleLines
Partner
StreamingTV
Dependents
OnlineBackup
PhoneService
DeviceProtection
gender
0.0
0.1
0.2
0.3
0.4
mean(|SHAP value|) (average impact on model output magnitude)




```
Top 5 Churn Drivers:

            Feature  Impact_Score
14         Contract      0.439554
7   InternetService      0.190408
4            tenure      0.137673
17   MonthlyCharges      0.078011
```

```
13  StreamingMovies      0.048218


 Highest-Risk Customers:

      Contract  MonthlyCharges  InternetService  Risk_Score
2493         0           90.75                1    0.953395
1976         0           93.55                1    0.953395
1704         0           99.75                1    0.953395
3727         0           96.60                1    0.953395
1880         0           86.00                1    0.953395
```

3.0.1 Business insights conclusion

These models are well optimized above the industry benchmark for recall and F1, if the business wants to prioritize catching churn customer, catboost weighted 5 would be their best option with sacrificng some precision and F1 score

If Telcom would like a more balanced approach, lightgbm weighted 5 would be their best option that optimize for both recall and F1 at around 70% and high ROC

[ ]:

```
```